Live Mainnet Governance Configuration Proof

1. High Level Conclusion

```
Branch 1 (the slow path, for real proposals)
  ProposerSafe (5/11) ------module------> Delay Modifier ------module------> OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol
  (+ ConfigLockGuard)                     (+ PauseGuard)                  (+ ConfigLockGuard)

Direct path (all nine OwnerlessSafe signers together: no delay, veto or pause)
  OwnerlessSafe (9/9) ------DEVOPS_ROLE------> Term Protocol

Branch 2 (the token-vote path, whose only power is to veto)
  Governor ------module------> Roles Modifier ------target------> DelayOwnerSafe ------owner------> Delay Modifier
                           (+ SetTxNonceGuard)                (+ ConfigLockGuard)

Pause controls
  PauseSafe (2/9) --pause-->               PauseGuard
  AdminSafe      --unpause / setPauser-->  PauseGuard
```

Trust assumption: Despite its name, the OwnerlessSafe has nine Term multisig holders as owners and a threshold of 9 (P1.4).

The conclusions drawn from these proofs establish the following propositions:

1. The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard, provided the DEVOPS_ROLE method call is not vetoed, and is executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.
2. Only TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, and Term multisig holders may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor, and 5. the txCooldown constant for the Delay Modifier.
3. Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the AdminSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.
4. The following: 1. the owner, target, modules, guard, role targets, role scopes, role assignments for the Roles Modifier, and 2. the voting token, proposal threshold, voting threshold for the Governor cannot be changed by anyone. The quorum for the Governor is 1% of TERM's total supply when a proposal is made; TERM's total supply can be changed only by the Term multisig holders on the ProposerSafe and OwnerlessSafe.
5. The following: 1. the owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the txCooldown, txExpiration for the Delay Modifier can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard for the OwnerlessSafe can be changed only by the Term

multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, and 2. the owner, target, modules, guard for the Delay Modifier cannot be changed by anyone.

6. The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The following: 1. the owners, fallbackHandler, modules, guard for the AdminSafe, and 2. the pauser for the PauseGuard can be changed only by the Term multisig holders on the AdminSafe. The ADMIN_ROLE holder for the PauseGuard cannot be changed by anyone.

Only Term multisig holders can execute DEVOPS_ROLE methods subject to TERM token governance, and this cannot be changed other than through Term multisig approval.

2. Evidence Classification

This document uses two classes of evidence to establish the security and functional properties of the governance configuration:

- **Formal Verification (FV) evidence** establishes that, assuming the stated configuration, the relevant contracts enforce the claimed security and functional properties. These properties are proved with the Certora Prover, a formal verification tool. For the guards (PauseGuard, ConfigLockGuard and SetTxNonceGuard), the proofs run on the contract code that was deployed, the code at commit `e9f3d20f4d16af7ab7a9fdf773e4fabd4cde3353` on `main`. For TERM, the TermToken rules (TT) and the Governor rules that read TERM run on the code the TERM proxy is to be upgraded to, the code at commit `74b19ca41aebf28c5b7801e94f082aad867b0f17` on term-token `main`. The proofs cover privileged entry points, authorized callers, configuration immutability, the absence of alternative execution paths, and the intended functionality. In the tables below, FV evidence is labeled **FV —** and names each Certora rule; ✅ marks a rule that passes.
- **Deployment evidence** establishes that the deployed contracts have the configuration assumed by the formal verification evidence. This includes on-chain verification of contract addresses, owners, modules, guards, thresholds, roles, and other relevant state, with each value read at a stated block. In the tables below, deployment evidence is labeled **On-chain —**; ✅ marks a read that matches the stated configuration.

These two classes of evidence are complementary. Formal verification evidence does not by itself establish that the deployed contracts have the configuration being proved, and deployment evidence does not by itself establish that the deployed configuration enforces the claimed security and functional properties.

Configuration items marked ⏳ (pending migration) describe the intended post-migration configuration and are not established by the current deployment evidence. These items must be re-verified against on-chain state after the migration is executed.

3. Evidence

Proposition 1 — The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe, and only they, may execute DEVOPS_ROLE methods, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and

ADMIN_ROLE holder constant for the PauseGuard, provided the DEVOPS_ROLE method call is not vetoed, and is executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.

#	Statement	Evidence
P1.1	Only the OwnerlessSafe may execute DEVOPS_ROLE methods.	**On-chain — The OwnerlessSafe is the only holder of the DEVOPS_ROLE AccessControl role required for DEVOPS_ROLE methods:** • The DEVOPS_ROLE methods are those listed in the DEVOPS_ROLE row of Term Finance's Access Controls page: `_authorizeUpgrade` on all contracts, `pauseDeploying` and `unpauseDeploying` on TermInitializer, `updateTreasuryAddress` and `updateProtocolReserveAddress` on TermController, and `addNewTokenPriceFeed` and `removeTokenPriceFeed` on TermPriceConsumerV3. • ✅ Each of the six Term Protocol contracts granted `DEVOPS_ROLE` to the OwnerlessSafe in its own creation transaction: ◦ TermController: block 25188635, tx `0xdeea44228ef14d20a66a02e0a3d84907bcd854aa0f352a5ad8233b48b23f7f15` ◦ TermEventEmitter: block 25188643, tx `0x15c3af7845f086fca8a069ea755f92f4613ae716978aa100089788d98a68d29e` ◦ TermPriceConsumerV3: block 25188652, tx `0x7b960b5c31408ac8cf6eb77766c2c8f747bf74b9f2ec1fd98f1d66999a3d6063` ◦ TermRepoDeployerFactory: block 25188657, tx `0x22c72baf2cf6a6ad808e874ca9d75190592059fa9e0591451ea738147394f8f5` ◦ TermInitializer: block 25555701, tx `0xec1c9e2935569e769123e7a05f6d1f5fc2c1ba7aa2a7b4013d799dfee76885d7` ◦ TermDiamond: block 25188499, tx `0xe70a739ab7ff78a6b8ae8beefd002c7ee4a87d8c41b6ec594ae45eba87412788` • ✅ In each contract's full Blockscout log history, that grant is the only event naming `DEVOPS_ROLE`. There is no `RoleRevoked`, no `RoleAdminChanged` and no grant to any other address. So the OwnerlessSafe still holds `DEVOPS_ROLE`, and nobody else ever has. • The other contracts with `_authorizeUpgrade` are proxies deployed against implementations, such as each term's TermAuction. Term's deploy system deploys every new proxy with `DEVOPS_ROLE` assigned only to the OwnerlessSafe. The deploy system runs off chain, so this is a deployment fact, not an on-chain read. **On-chain — Nobody can grant DEVOPS_ROLE to another address that is not the OwnerlessSafe:** • ✅ On TermController, TermEventEmitter, TermPriceConsumerV3, TermInitializer and TermRepoDeployerFactory, `getRoleAdmin(DEVOPS_ROLE)` is `DEFAULT_ADMIN_ROLE` (`0x00`). So only a holder of `DEFAULT_ADMIN_ROLE` can call `grantRole(DEVOPS_ROLE, …)`. • ✅ No `RoleGranted` for `DEFAULT_ADMIN_ROLE` appears in any of the six contracts' histories, so nobody holds it. So nobody can grant `DEVOPS_ROLE` on those five contracts.
P1.2	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `requiredTxGas`, `simulateAndRevert`, `fallback` and `receive`:** • SE-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: GnosisSafe v1.3.0 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `requiredTxGas`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), and any unsupported call lands in `fallback`. Each blob below covers some of these entry points, and together they cover all seventeen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Safe:** • SE-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that

#	Statement	Evidence
		owner's signature in `execTransaction`. **FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved:** • SE-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE-8 ensures the owner recorded it itself. So with SE-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler` are only callable from Modules of Safe or Owners of Safe:** • SE-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard and fallbackHandler only change when the caller is the Safe itself. So a call to any of these eight functions from anyone else changes nothing. • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. A fallbackHandler is treated as the Safe itself, so it cannot reach these functions even then. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of Owners of Safe. • SC-1 `moduleCanChangeSettings`, SC-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its `execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting, once the threshold is passed. Together with SC-1 and SC-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The OwnerlessSafe is set up: its threshold is 9 and it has nine owners (P1.4). **FV — `requiredTxGas` and `simulateAndRevert` always revert:** • SE-10 `requiredTxGasAlwaysReverts`, SE-11 `simulateAndRevertAlwaysReverts` ✅ pass: anyone can call them, and they do execute a call (`requiredTxGas`) or delegatecall (`simulateAndRevert`), but they always revert afterwards, so none of their effects persist. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE-23 `storageHookFires`, SE-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE-21's checks do detect storage writes and calls. • SE-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE-21 is not vacuous. **FV — `receive` is callable by anyone but does not change state:** • SE-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It

#	Statement	Evidence
		writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE-23 `storageHookFires`, SE-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE-25's checks do detect storage writes and calls. • SE-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE-25 is not vacuous. **On-chain — The OwnerlessSafe runs GnosisSafe v1.3.0, the code the SE and SC rules check:** • ✅ At block 26100998, storage slot 0 (the proxy's `singleton`) of the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) holds `0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`, whose `VERSION()` returns `1.3.0`. • So the rules above, which check GnosisSafe v1.3.0, hold for the OwnerlessSafe.
P1.3	OwnerlessSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • ✅ The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:12`) on the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) is `address(0)` at block 26092959.
P1.4	`Owners of OwnerlessSafe = Term multisig holders on OwnerlessSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — Owners of OwnerlessSafe are these nine Term multisig holders, and all nine must approve each transaction:** ✅ At block 26084853, the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) runs GnosisSafe v1.3.0 (`VERSION`), has threshold 9 (`getThreshold`) and these 9 owners (`getOwners`), each an externally owned account with no contract code: • `0xc1047a4D9f6071B55585523F7F659F39A1fBA74b` • `0x8EF485fD38b7B29a827938C9F00300eaBf8E1710` • `0xbfFcAdCd5549cC378693108BcD4435776A6fa795` • `0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3` • `0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60` • `0xE82183cfAE1C24f044315c318FD97bE7e47b31D2` • `0xeee661edcFE634Dc0e29D62C26AfD62c0843b817` • `0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81` • `0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D` • The identities of the nine key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P1.5	`Modules of OwnerlessSafe = Delay Modifier`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • ✅ The OwnerlessSafe's (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) module list returns just one, the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`), and the list ends there (`getModulesPaginated`, block 26092962).
P1.6	Only the Modules of the Delay Modifier and the Owner of the Delay Modifier can interact with the Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip	**FV — Callers can only interact with the Delay Modifier through `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`:** • DM-13 `delayWriteFunctionsAreTheKnownFifteen` ✅ passes: the Delay Modifier has no fallback and no write functions besides `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`,

#	Statement	Evidence
	transactions queued by the Modules of the Delay Modifier or the Owner of the Delay Modifier.	`transferOwnership`, `renounceOwnership`, `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `executeNextTx`, `skipExpired` and `setUp`. Each blob below covers some of these, and together they cover all fifteen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Delay Modifier:** • DM-20 `onlyEnabledModulesCanCallExecFromModule` ✅ passes: only an enabled module (Modules of Delay Modifier) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds. So DM-20 is not vacuous. **FV — `setTxCooldown`, `setTxExpiration`, `setTxNonce`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership` are only callable by Owner of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of these ten functions succeeds only when the caller is the owner (Owner of Delay Modifier), and from the same state no two different callers can both succeed. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from the owner succeeds. So DM-8 is not vacuous. **FV — `setUp` always reverts after deploy:** • DM-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: nobody can re-run the Delay Modifier's setup after deployment. **FV — `executeNextTx` and `skipExpired` are callable by anyone, but can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier:** • DM-11 `anyoneCanExecuteNextTx`, DM-12 `anyoneCanSkipExpired` ✅ pass: whether a call to `executeNextTx` or `skipExpired` succeeds, and what it changes, does not depend on who the caller is. • DM-17 `outsiderCanExecuteNextTx`, DM-18 `outsiderCanSkipExpired` ✅ pass: a caller that is neither the owner nor a module can execute or skip a queue entry. So DM-11 and DM-12 are not vacuous. • DQ-2 `everyQueuedEntryWasQueuedByModuleOrOwner` ✅ passes: every transaction in the queue was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-3 `executeNextTxRunsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: `executeNextTx` only runs the queue entry at `txNonce`. By DQ-2, that entry was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-4 `skipExpiredSkipsOnlyEntriesQueuedByModuleOrOwner` ✅ passes: every index `skipExpired` steps over is a queue entry, and by DQ-2 it was queued by Modules of Delay Modifier or Owner of Delay Modifier. • DQ-5 `executeNextTxCanRunAQueuedEntry`, DQ-6 `skipExpiredCanSkipAQueuedEntry` ✅ pass: `executeNextTx` can run a queue entry, and `skipExpired` can skip one. So DQ-3 and DQ-4 are not vacuous.
P1.7	Only the Modules of the Delay Modifier can queue transactions to the Delay Modifier. The Owner of the Delay Modifier, the DelayOwnerSafe, cannot queue transactions to the Delay Modifier.	**FV — Only Modules of Delay Modifier can queue transactions to the Delay Modifier:** • DM-4 `queueOnlyGrowsThroughEnabledModules` ✅ passes: over every write function, only an enabled module (Modules of Delay Modifier) can move the queue nonce or write a queue entry. • DM-5 `enabledModuleCanQueue` ✅ passes: an enabled module's call to `execTransactionFromModule` succeeds and moves the queue nonce forward by one. So DM-4 is not vacuous. **On-chain — Owner of Delay Modifier, the DelayOwnerSafe, is not a Module of Delay Modifier:** • ✅ At block 26135270, `owner()` on the Delay Modifier returns the DelayOwnerSafe (P2.3). • ✅ At block 26084775, `isModuleEnabled` on the Delay Modifier returns false for the DelayOwnerSafe. This matches P1.8: the Delay Modifier's only module (Modules of Delay Modifier) is the ProposerSafe. • So, since the DelayOwnerSafe is the owner, by DM-4 the Owner of Delay Modifier cannot queue transactions.

#	Statement	Evidence
P1.8	`Modules of Delay Modifier = ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • ✅ The Delay Modifier's (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) module list returns just one, the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`), and reports that the list ends there (`getModulesPaginated`, block 26080101).
P1.9	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2. **On-chain — The ProposerSafe runs the same GnosisSafe v1.3.0 code as the OwnerlessSafe, so P1.2's evidence applies to the ProposerSafe:** • ✅ At block 26100998, storage slot 0 (the proxy's `singleton`) of the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) and of the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) both hold `0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`, whose `VERSION()` returns `1.3.0`. • ✅ Both proxies also have the same runtime code (code hash `0xb89c1b3bdf2cf8827818646bce9a8f6e372885f8c55e5c07acbd307cb133b000`). • So both Safes run the same proxy in front of the same mastercopy, and P1.2's rules, which check GnosisSafe v1.3.0, hold for the ProposerSafe too.
P1.10	ProposerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • ✅ The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:12`) on the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) is `address(0)` at block 26056133.
P1.11	`Owners of ProposerSafe = Term multisig holders on ProposerSafe`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner, target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — Owners of ProposerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** ✅ At block 26084863, the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) runs GnosisSafe v1.3.0 (`VERSION`), has threshold 5 (`getThreshold`) and these 11 owners (`getOwners`), each an externally owned account with no contract code: • `0xF6A745f9B38FFcd17Ee8909AC89151D788F95282` • `0xC43d527E3544A3d199Ce28E87994384D89d90e6E` • `0x8EF485fD38b7B29a827938C9F00300eaBf8E1710` • `0xbfFcAdCd5549cC378693108BcD4435776A6fa795` • `0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3` • `0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60` • `0xE82183cfAE1C24f044315c318FD97bE7e47b31D2` • `0xeee661edcFE634Dc0e29D62C26AfD62c0843b817` • `0xc1047a4D9f6071B55585523F7F659F39A1fBA74b` • `0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81` • `0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D` • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P1.12	`Modules of ProposerSafe = null`, holding: 1. the owners, fallbackHandler, modules, and guard constant for the ProposerSafe and OwnerlessSafe; 2. the owner,	**On-chain — The Modules of ProposerSafe list is empty:** • ✅ `getModulesPaginated` on the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) returns an empty list at block 26050570.

#	Statement	Evidence
	target, modules, guard, txCooldown, and txExpiration constant for the Delay Modifier; and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	
P1.13	The Term multisig holders on the ProposerSafe and the Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods, provided the DEVOPS_ROLE method call is not vetoed, and is executed while the PauseGuard is not paused and before the 24-hour expiration period following the cooldown ends.	**FV — The Term multisig holders on the ProposerSafe can execute DEVOPS_ROLE methods by proposing them to the Delay Modifier and executing them after the cooldown, provided they are not vetoed, while the PauseGuard is not paused and before the proposal expires:** • DE-1 `proposerOwnersExecuteDevopsMethodThroughTheDelay` ✅ passes. In one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(<protocol contract>, 0, <DEVOPS_ROLE method call>, Call)`. ◦ Anyone submits it through `execTransaction` with that approval as the signature, so the Delay Modifier queues the proposal. ◦ Nobody vetoes it. Once the cooldown has passed, before the proposal expires, and with the PauseGuard installed and not paused, anyone's `executeNextTx` hands it to the OwnerlessSafe. ◦ The OwnerlessSafe calls the protocol contract. The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE-1 sets the ProposerSafe's threshold to 1, so one owner's approval is the whole threshold. SE-6 `ownersCanMakeTheSafeAct` (P1.2) ✅ passes for any threshold, so with the real threshold of 5 (P1.11), five owners' approvals take the same path. • DE-1 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier (P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard. DE-1 also assumes neither Safe has a guard; each has a ConfigLockGuard, which the next two bullets cover. The on-chain blob below covers the parts not shown elsewhere. • The ProposerSafe's ConfigLockGuard lets the proposal through. Its `lockedModifier` is the Delay Modifier (P5.DelayModifier.13), so the proposal, `execTransactionFromModule` on the Delay Modifier, is a plain call to the locked modifier. It is not one of the seven calls the guard locks there (CL141-6 to CL141-9). CL141-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedModifier` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:144`), so DE-1's ProposerSafe step is unchanged. • The OwnerlessSafe's ConfigLockGuard never checks the Delay Modifier's call. CL-18 `moduleCanDelegateCallPastTheGuard`, CL-19 `moduleCanDelegateCallPastTheGuardWithReturnData` and CL-20 `moduleCanRemoveTheGuard` ✅ pass: with ConfigLockGuard installed, a module's delegate call still succeeds, and a module can still move the guard out of its slot. The guard rejects both on the signed path, so it is not consulted on the module path. LG-3 `moduleCanStillCallOut` ✅ passes: with ConfigLockGuard installed and its `lockedModifier` the Delay Modifier (P5.DelayModifier.13), a plain `execTransactionFromModule` from the Delay Modifier, as an enabled module, to another contract succeeds. • DE-1 also assumes the PauseGuard is not paused. While it is paused, the PauseGuard rejects every `executeNextTx` (`PauseGuard.sol:116`), so the proposal waits until a holder of the PauseGuard's `ADMIN_ROLE` unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **On-chain — The setup formally verified above is deployed, and the Delay Modifier's expiration period is 24 hours:** • ✅ At block 26114802, `target()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) returns the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`). • ✅ At block 26135270, `guard()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) returns the PauseGuard (`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`). • ✅ At block 26114802, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30`) on the OwnerlessSafe and on the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) holds the ConfigLockGuard (`0xeda83f6d7a8a56d4aa5433bf7cc1fef496036d53`). • ✅ At block 26114802, `paused()` on the PauseGuard returns false. • ✅ At block 26229805, `txExpiration()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) is 86,400 seconds (24

#	Statement	Evidence
		hours). **FV — The Term multisig holders on the OwnerlessSafe may execute DEVOPS_ROLE methods directly from the OwnerlessSafe:** • DE-2 `ownerlessOwnersExecuteDevopsMethodThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the OwnerlessSafe (a Term multisig holder, P1.4) approves with `approveHash` an OwnerlessSafe transaction that calls a DEVOPS_ROLE method on a protocol contract. ◦ Anyone submits it through `execTransaction` with that approval as the signature. ◦ The call passes `onlyRole(DEVOPS_ROLE)` because the OwnerlessSafe holds `DEVOPS_ROLE` (P1.1), and the method runs. • DE-2 sets the OwnerlessSafe's threshold to 1. By SE-6 (P1.2), with the real threshold of 9 (P1.4), all nine owners' approvals take the same path. • DE-2 assumes the OwnerlessSafe has no guard. Its ConfigLockGuard lets the call through. Its `lockedModifier` is the Delay Modifier (P5.DelayModifier.13), and a DEVOPS_ROLE method call to a protocol contract is a plain call to neither the OwnerlessSafe nor the Delay Modifier, so it is not a locked call. CL141-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedModifier` is, the Delay Modifier included, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:144`), so DE-2's path is unchanged.

Proposition 2 — Only TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, and Term multisig holders may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor, and 5. the txCooldown constant for the Delay Modifier.

#	Statement	Evidence
P2.1	Only Modules of Delay Modifier and Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P2.2	Veto Delay Modifier transactions (`Delay.setTxNonce(uint256)`) is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier.	**FV — `setTxNonce`, which vetoes Delay Modifier transactions, is only callable by Owner of Delay Modifier, and not by Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: `setTxNonce` is one of the Delay Modifier's ten `onlyOwner` functions, and each succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-15 `enabledModuleThatIsNotOwnerCannotSetTxNonce` ✅ passes: in particular, an enabled module (Modules of Delay Modifier) that is not the owner cannot call `setTxNonce`; the call always reverts, whatever the nonce. By P1.7, the owner is not a module, so this covers every module. • DM-16 `ownerCanSetTxNonce` ✅ passes: the owner's `setTxNonce` succeeds and sets `txNonce`. So DM-8 and DM-15 do not hold just because `setTxNonce` always reverts.
P2.3	`Owner of Delay Modifier = DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard	**FV — The Owner of Delay Modifier is the single address `owner()` returns:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions succeeds only when the

#	Statement	Evidence
	constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	caller is the address `owner()` returns, and from the same state no two different callers can both succeed. So only one address at a time can act as the Owner of Delay Modifier. • DM-19 `ownerCanCallEachDelaySetting` ✅ passes: for each of the ten, some call from that address succeeds. So DM-8 is not vacuous. **On-chain — `owner()` on the Delay Modifier returns the DelayOwnerSafe:** • ✅ At block 26135270, `owner()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) returns the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`).
P2.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	**FV — Callers can only interact with the Safe through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `simulateAndRevert`, `fallback` and `receive`:** • SE141-1 `safeEntryPointsAreAllAccountedFor` ✅ passes: GnosisSafe v1.4.1 has no write functions besides `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransaction`, `approveHash`, `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard`, `setFallbackHandler`, `setup`, `simulateAndRevert`, `fallback` and `receive` (the Prover checks `fallback` and `receive` as one entry), and any unsupported call lands in `fallback`. These are the seventeen entry points in P1.2 minus `requiredTxGas`, which Safe removed in v1.4.0. Each blob below covers some of these entry points, and together they cover all sixteen. **FV — `execTransactionFromModule` and `execTransactionFromModuleReturnData` are only callable by Modules of Safe:** • SE141-3 `onlyEnabledModulesCanCallModuleExec` ✅ passes: only an enabled module (Modules of Safe) can successfully call `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • SE141-4 `enabledModuleCanMakeTheSafeAct` ✅ passes: an enabled module can call them directly, with no owner signatures, and the Safe makes an outgoing call or delegatecall. So SE141-3 is not vacuous. **FV — `approveHash` is only callable by Owners of Safe:** • SE141-8 `onlyOwnersCanApproveHashes` ✅ passes: in any call, an approval is only recorded when the caller is an owner (Owners of Safe) recording its own approval. So a call to `approveHash` from anyone else records nothing. An owner's recorded approval then counts as that owner's signature in `execTransaction`. **FV — `execTransaction` is callable by anyone, but only executes once a threshold of Owners of Safe have approved:** • SE141-5 `execTransactionRunsOnlyAfterTheSignatureCheck` ✅ passes: whoever the caller is, `execTransaction` only runs once the signature check has passed for that exact transaction, asking for as many signatures as the threshold. The rule places no restriction on the caller. • SE141-12 `eachSignatureAcceptsANewApprovingOwner` ✅ passes: each signature the check accepts comes from a different owner (Owners of Safe) who approved the transaction. For an approval made through `approveHash`, SE141-8 ensures the owner recorded it itself. So with SE141-5, the Safe acts through `execTransaction` only once as many different owners as the threshold have approved. • SE141-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners (Owners of Safe). So SE141-5 is not vacuous. **FV — `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler` are only callable from Modules of Safe or Owners of Safe:** • SE141-7 `settingsOnlyChangeWhenTheSafeCallsItself` ✅ passes: in any call, the owners, threshold, modules, guard and fallbackHandler only change when the caller is the Safe itself. So a call to any of these eight functions from anyone else changes nothing. • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, a setting only changes through an enabled module's (Modules of Safe) `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. A fallbackHandler is treated as the Safe itself, so it cannot reach these functions even then. By the `execTransactionFromModule` and `execTransaction` blobs above, those paths are open only to Modules of Safe and to a threshold of

#	Statement	Evidence
		Owners of Safe. • SC141-1 `moduleCanChangeSettings`, SC141-2 `moduleCanChangeSettingsWithReturnData` ✅ pass: for each function above, a module's (Modules of Safe) `execTransactionFromModule`, and likewise its `execTransactionFromModuleReturnData`, addressed to the Safe with a call to that function, succeeds and changes a setting. • SC141-3 `ownersCanChangeSettings` ✅ passes: for each function above, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. Together with SC141-1 and SC141-2, this shows the restriction is not achieved by nothing working. **FV — `setup` always reverts after deploy:** • SE141-9 `setupAlwaysRevertsAfterSetup` ✅ passes: once the Safe is set up, `setup` always reverts, so nobody can re-run it to replace the owners, threshold, modules or handler. • The DelayOwnerSafe is set up: its threshold is 5 and it has eleven owners (P2.6). **FV — `simulateAndRevert` always reverts:** • SE141-11 `simulateAndRevertAlwaysReverts` ✅ passes: anyone can call `simulateAndRevert`, and it does make a delegatecall, but it always reverts afterwards, so none of its effects persist. **FV — `fallback` is callable by anyone but does not change state, assuming the fallbackHandler is `address(0)`:** • SE141-21 `noHandlerFallbackChangesNoState` ✅ passes: with no handler set, `fallback` changes no state at all. It writes no storage, makes no call of any kind (call, delegatecall, staticcall or callcode) and creates or destroys no contract, and a `fallback` call that succeeds carries no ETH, so not even the Safe's balance changes. • SE141-23 `storageHookFires`, SE141-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE141-21's checks do detect storage writes and calls. • SE141-22 `noHandlerFallbackSucceeds` ✅ passes: with no handler set, some call to `fallback` succeeds, so SE141-21 is not vacuous. **FV — `receive` is callable by anyone but does not change state:** • SE141-25 `receiveChangesNoState` ✅ passes: whatever the handler, a call that runs `receive` changes no state beyond the ETH it is sent. It writes no storage, makes no call of any kind and creates or destroys no contract. • Here, state means the Safe's storage, which holds its owners, threshold, modules, guard, fallbackHandler, nonce and approvals. The ETH sent is added to the Safe's balance, which is not part of that storage. • SE141-23 `storageHookFires`, SE141-24 `callHookFires` ✅ pass: a successful `approveHash` is seen writing the Safe's storage, and `fallback` with a handler set is seen making its call. So SE141-25's checks do detect storage writes and calls. • SE141-26 `receiveRuns` ✅ passes: some call to the Safe runs `receive` and succeeds, so SE141-25 is not vacuous. **On-chain — The DelayOwnerSafe runs GnosisSafe v1.4.1, the code the SE141 and SC141 rules check:** • ✅ At block 26084873, `VERSION()` on the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) returns `1.4.1` (P2.6). • So the evidence above is P1.2's, with each rule run against GnosisSafe v1.4.1 instead of the v1.3.0 the OwnerlessSafe and ProposerSafe run (P1.9).
P2.5	DelayOwnerSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • ✅ The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:14` in GnosisSafe v1.4.1) on the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) is `address(0)` at block 26080124.

#	Statement	Evidence
P2.6	`Owners of DelayOwnerSafe = Term multisig holders on DelayOwnerSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** ✅ At block 26084873, the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) runs GnosisSafe v1.4.1 (`VERSION`), has threshold 5 (`getThreshold`) and these 11 owners (`getOwners`), each an externally owned account with no contract code: • `0xeee661edcFE634Dc0e29D62C26AfD62c0843b817` • `0xF6A745f9B38FFcd17Ee8909AC89151D788F95282` • `0xC43d527E3544A3d199Ce28E87994384D89d90e6E` • `0x8EF485fD38b7B29a827938C9F00300eaBf8E1710` • `0xbfFcAdCd5549cC378693108BcD4435776A6fa795` • `0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3` • `0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60` • `0xE82183cfAE1C24f044315c318FD97bE7e47b31D2` • `0xc1047a4D9f6071B55585523F7F659F39A1fBA74b` • `0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81` • `0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D` • The identities of the eleven key holders are not visible on chain. Their status as Term multisig holders is a deployment fact.
P2.7	`Modules of DelayOwnerSafe = Roles Modifier`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • ✅ The DelayOwnerSafe's (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) module list returns just one, the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`), and the list ends there (`getModulesPaginated`, block 26080124).
P2.8	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	**FV — Callers can only interact with the Roles Modifier through `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole`, `execTransactionWithRoleReturnData`, `setMultisend`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles`, `setDefaultRole`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership`, `renounceOwnership` and `setUp`:** • RI-14 `rolesWriteFunctionsAreTheKnownTwentyFive` ✅ passes: the Roles Modifier has no `fallback` or `receive` and no write functions besides these twenty-five. Each blob below covers some of these, and together they cover all twenty-five. **FV — `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData` allow only Roles Modifier Modules with AssignedRole to execute transactions through Roles Modifier:** • RI-7 `onlyEnabledModulesCanExec` ✅ passes: only enabled modules can call `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData`. • RI-8 `moduleWithoutDefaultRoleCannotExecFromModule` ✅ passes: an enabled module that is not a member of its default role cannot execute through `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • RI-9 `moduleWithoutRoleCannotExecTransactionWithRole`, RI-10 `moduleWithoutRoleCannotExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is not a member of the role the call names cannot execute through `execTransactionWithRole` or `execTransactionWithRoleReturnData`. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 `roleMemberModuleCanExecFromModule`, RI-12 `roleMemberModuleCanExecTransactionWithRole`, RI-13 `roleMemberModuleCanExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is a member of the role the call runs under (Roles Modifier Modules with AssignedRole) can successfully call each of the four, so the restriction is not achieved by nothing working.

#	Statement	Evidence
		FV — `setMultisend`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles`, `setDefaultRole`, `setAvatar`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership` are only callable by Roles Modifier Owner: • RI-5 `onlyOwnerCanCallRolesSettings` ✅ passes: each of these twenty functions succeeds only when the caller is the owner (Roles Modifier Owner). • RI-2 `rolesConfigOnlyChangesThroughOwner` ✅ passes: over every write function, the four execution functions included, the owner, guard, multisend, avatar, target, module list, default roles, role membership and target clearances only change when the caller is the owner. So no other entry point offers a way around RI-5. • RI-6 `ownerCanCallEachRolesSetting` ✅ passes: for each of the twenty, some call from the owner succeeds. So RI-5 is not vacuous. **FV — `setUp` always reverts after deploy:** • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: once the module list is set up, `setUp` always reverts, so nobody can re-run it to replace the owner or the module list. • The Roles Modifier's module list is set up: it holds the Governor (P2.10).
P2.9	Only Roles Modifier Modules with AssignedRole can execute transactions through Roles Modifier. Roles Modifier Owner cannot do so.	**FV — `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData` allow only Roles Modifier Modules with AssignedRole to execute transactions through Roles Modifier:** • RI-7 `onlyEnabledModulesCanExec` ✅ passes: only enabled modules can call `execTransactionFromModule`, `execTransactionFromModuleReturnData`, `execTransactionWithRole` and `execTransactionWithRoleReturnData`. • RI-8 `moduleWithoutDefaultRoleCannotExecFromModule` ✅ passes: an enabled module that is not a member of its default role cannot execute through `execTransactionFromModule` or `execTransactionFromModuleReturnData`. • RI-9 `moduleWithoutRoleCannotExecTransactionWithRole`, RI-10 `moduleWithoutRoleCannotExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is not a member of the role the call names cannot execute through `execTransactionWithRole` or `execTransactionWithRoleReturnData`. With RI-8, a module with no assigned role can execute nothing through any of the four. • RI-11 `roleMemberModuleCanExecFromModule`, RI-12 `roleMemberModuleCanExecTransactionWithRole`, RI-13 `roleMemberModuleCanExecTransactionWithRoleReturnData` ✅ pass: an enabled module that is a member of the role the call runs under (Roles Modifier Modules with AssignedRole) can successfully call each of the four, so the restriction is not achieved by nothing working. **FV — Roles Modifier Owner cannot execute transactions through Roles Modifier unless it is one of the Roles Modifier Modules with AssignedRole:** • RI-22 `ownerWithoutModuleRoleCannotExecFromModule`, RI-23 `ownerWithoutModuleRoleCannotExecTransactionWithRole`, RI-24 `ownerWithoutModuleRoleCannotExecTransactionWithRoleReturnData` ✅ pass: on all four execution entry points, the owner's (Roles Modifier Owner) call reverts unless the owner is itself an enabled module and a member of the role the call runs under (its default role for the two FromModule functions, the role it names for the two WithRole functions). **On-chain — Roles Modifier Owner is not also a Roles Modifier Module:** • ✅ At block 26101471, `owner()` on the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) returns `address(0)`, and the `_owner` slot (slot 51, `OwnableUpgradeable`) holds zero. The OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) renounced ownership at block 26101459 (`OwnershipTransferred` to `address(0)`, tx `0xccd4324dd062785a3a4c8247268b89f5d10ad73b318abf7bcb7f4486c7add46a`), through a proposal run by the Delay Modifier's `executeNextTx`. • `address(0)` can never be an enabled module: `enableModule` rejects it (zodiac `Modifier.sol:87`). ✅ At the same block, `isModuleEnabled(address(0))` returns false, and the module list holds only the Governor

#	Statement	Evidence
		(`0x2B715634134220ffeEE9458b4e34E41A41418607`). • So the owner is not a module, and by RI-22 to RI-24 it cannot execute transactions through the Roles Modifier.
P2.10	`Roles Modifier Modules with AssignedRole = Governor`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Roles Modifier Modules list contains only the Governor:** • ✅ The Roles Modifier's (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) module list returns just one, the Governor (`0x2B715634134220ffeEE9458b4e34E41A41418607`), and the list ends there (`getModulesPaginated`, block 26114992). • ✅ The Roles Modifier's complete event history, from its creation at block 26042039 to block 26114992, agrees: one `EnabledModule`, for the Governor, and no `DisabledModule`. **On-chain — The Governor is a Roles Modifier Module with an AssignedRole:** • ✅ The Governor is recorded as a member of Role 1 (`roles[1].members`, storage slot 107, block 26114992). • So the Governor is the only Roles Modifier Module, and it has an assigned role: Roles Modifier Modules with AssignedRole = Governor.
P2.11	Governor is assigned only Role 1, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**FV — A Roles Modifier Module's roles change only through `assignRoles`, and its default role only through `setDefaultRole`, and each change emits an event:** • RI-19 `membershipOnlyChangesThroughAssignRoles` ✅ passes: over every write function, `setUp` included, and any caller, a Roles Modifier Module's membership of a role changes only through `assignRoles` (`Roles.sol:290`), and every change emits `AssignRoles`. • RI-20 `defaultRoleOnlyChangesThroughSetDefaultRole` ✅ passes: likewise, a module's default role changes only through `setDefaultRole` (`Roles.sol:310`), and every change emits `SetDefaultRole`. **On-chain — Governor is assigned only Role 1:** • ✅ The Roles Modifier's complete event history, from its creation at block 26042039 to block 26114992, has exactly one `AssignRoles`: the Governor, Role 1, member (tx `0x1d4ec5029b9c66cff2c92bc8be16196bba4d4484caaa74488bdcc61616fb186a`). By RI-19, that is every membership change since creation, so the Governor is a member of Role 1 and of no other role. • ✅ The same history has exactly one `SetDefaultRole`: the Governor, Role 1 (tx `0xf01e95980c86dc59667c78094ba1f7a65c744b5214029ef70e721ec8c340a928`). By RI-20, the Governor's default role is Role 1, so its `execTransactionFromModule` calls also run under Role 1.
P2.12	Only Holders of Governor Vote Token can interact with the Governor, except `execute`, which anyone can call only if Holders of Governor Vote Token proposed and passed the proposal, and except the `castVote` functions (`castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`), which anyone can call but which count a non-holder's vote as 0, and except `receive`, `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived`, which anyone can call but which make no storage writes.	**FV — Callers can only interact with the Governor through `propose`, `queue`, `execute`, `cancel`, `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`, `relay`, `onERC721Received`, `onERC1155Received`, `onERC1155BatchReceived` and `receive`:** • GS-1 `governorWriteFunctionsAreTheKnownFourteen` ✅ passes: the Governor has no write functions besides `propose`, `queue`, `execute`, `cancel`, `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`, `relay`, `onERC721Received`, `onERC1155Received`, `onERC1155BatchReceived` and `receive`. Each blob below covers some of these, and together they cover all fourteen. **FV — `propose` is only callable by Holders of Governor Vote Token with at least the proposal threshold of voting power:** • Holders of Governor Vote Token are the source of all voting power in the Governor, including the power a delegate votes with. The Governor counts TERM's `getPastVotes` (GV-4, P2.13). TERM is an `ERC20VotesUpgradeable` token (`TermToken.sol:20`), so a holder votes with its own TERM by delegating to itself, or delegates it to another address. A delegate then votes with the holders' TERM: its power comes only from the holders who delegated to it, and they can move it elsewhere at any time. So every vote, and every proposal over the threshold, rests on TERM that holders chose to delegate. • GP-1 `callerBelowProposalThresholdCannotPropose` ✅ passes: any caller whose voting power at `clock() - 1` is below the proposal threshold cannot create a proposal. • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: the proposal threshold is fixed at 1,000 tokens in the Governor's code. It is above zero, so by GP-1 no non-holder can propose. • GV-1 `holderAboveThresholdCanProposeVeto` ✅ passes: a caller with at least the proposal threshold of voting power can propose the veto. So GP-1 is not vacuous.

#	Statement	Evidence
		FV — `cancel` is only callable by the proposer, who is a Holder of Governor Vote Token: • GS-5 `onlyTheProposerCanCancel` ✅ passes: a successful `cancel` came from the address recorded as the proposal's proposer. By GP-1, only Holders of Governor Vote Token can have proposed. • GS-6 `proposerCanCancel` ✅ passes: the proposer can cancel, so GS-5 is not vacuous. **FV — The `castVote` functions are callable by anyone, but a non-holder's vote counts as 0:** • GC-1 `directVoteCountsOnlyTheVotersVotes`, GC-2 `voteBySigCountsOnlyTheVotersVotes`, GC-3 `voteWithReasonAndParamsBySigCountsOnlyTheVotersVotes` ✅ pass: through each of `castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig` and `castVoteWithReasonAndParamsBySig`, a proposal's tally grows by exactly the voter's votes at its snapshot, so a caller with none moves nothing. • GC-4 `nothingButAVoteMovesATally` ✅ passes: no other write function moves a tally. • GC-5 `termHolderVoteCounts` ✅ passes: a caller with votes at the snapshot can vote, and its vote moves the tally. So GC-1 to GC-3 are not vacuous. • As in the `propose` blob, all voting power comes from holders' TERM. A caller that holds no TERM and has none delegated to it has no votes, so its vote counts as 0. **FV — `queue` never succeeds:** • GS-4 `queueAlwaysReverts` ✅ passes: every call to `queue` reverts, so it never sets a proposal's eta. **FV — `execute` is callable by anyone, but only executes proposals Holders of Governor Vote Token proposed and passed:** • GS-10 `executeOnlyRunsSucceededProposals` ✅ passes: a successful `execute` ran a proposal whose state was `Succeeded`. • GS-9 `noProposalIsQueued` ✅ passes: no proposal is ever `Queued`, the other state `execute` accepts, because the only write to a proposal's eta is in `queue`, which never succeeds (GS-4). • So `execute` only runs a proposal created by a Holder of Governor Vote Token with at least the proposal threshold of voting power (GP-1), and passed on Governor Vote Token votes (GC-1 to GC-4). • GV-2 `passedVetoReachesTheDelayOwnerSafe` ✅ passes: a proposal in state `Succeeded` executes. So GS-10 is not vacuous. **FV — `relay` can only run as an action of an executed proposal, so only a passed proposal by Holders of Governor Vote Token can reach it:** • GS-11 `relayOnlyCallableByTheGovernor` ✅ passes: a successful `relay` came from the Governor itself. • GS-15 `governorOnlyCallsItselfFromExecuteAndRelay` ✅ passes: no write function except `execute` and `relay` makes a call to the Governor, so the Governor only calls itself as an action of an executed proposal. • By the `execute` blob above, an executed proposal was proposed and passed by Holders of Governor Vote Token. • GS-12 `relayCanRun` ✅ passes: the Governor can relay, so GS-11 is not vacuous. **FV — `receive` is callable by anyone but makes no storage writes:** • GS-7 `receiveChangesNoState` ✅ passes: a successful `receive` writes no storage, makes no call of any kind and creates or destroys no contract. • GS-8 `receiveRuns` ✅ passes: some call to `receive` succeeds, so GS-7 is not vacuous. **FV — `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived` are callable by anyone but make no storage writes:** • GS-13 `tokenReceiptChangesNoState` ✅ passes: a successful call to any of the three writes no storage, makes no call of any kind and creates or destroys no contract. • GS-14 `tokenReceiptRuns` ✅ passes: each can succeed, so GS-13 is not vacuous.

#	Statement	Evidence
P2.13	`Governor Vote Token = TERM token`, holding: 1. the owners, fallbackHandler, modules, guard constant for the DelayOwnerSafe, and 2. the owner, target, modules, guard constant for the Delay Modifier and Roles Modifier, and 3. the role targets, role scopes, role assignments constant for the Roles Modifier, and 4. the voting token, proposal threshold, voting threshold, quorum constant for the Governor.	**On-chain — The Governor's vote token is the TERM token:** • ✅ At block 26095237, `token()` on the Governor (`0x2B715634134220ffeEE9458b4e34E41A41418607`) returns the TERM token (`0xC3d21f79C3120A4fFda7A535f8005a7c297799bF`). **FV — The Governor counts votes in that token:** • GV-4 `governorVotesAreTermPastVotes` ✅ passes: for any account and any past timepoint, the Governor's vote count is its vote token's `getPastVotes`. So the Governor Vote Token is the TERM token.
P2.14	The TERM token holders, through a proposal made by a holder with at least 1,000 TERM of voting power that reaches a quorum of 1,000,000 TERM, and the Term multisig holders on the DelayOwnerSafe may veto Delay Modifier transactions within the 24-hour veto window, in which a Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued.	**FV — A Delay Modifier transaction cannot be executed until txCooldown has passed since it was queued:** • DP-34 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. So `executeNextTx` is the only way a queued transaction is executed. • DP-28 `executeNextTxConsumesOnlyTheEntryAtTxNonce` ✅ passes: `executeNextTx` only executes the queue entry at `txNonce`, and moves `txNonce` forward by one. • DP-30 `executeNextTxRevertsDuringCooldown` ✅ passes: until `txCooldown` has passed since the entry at `txNonce` was queued, `executeNextTx` always reverts, whoever calls it and whatever it is given. With DP-28, no queue entry can be executed before its own cooldown has passed. This is the veto window. • DP-29 `queuedTransactionExecutesAfterCooldown` ✅ passes: once the cooldown has passed, before the entry expires and with the PauseGuard not paused, anyone's `executeNextTx` executes it. So DP-30 is not vacuous, and the veto window ends when the cooldown does. • `executeNextTx` compares against `txCooldown` as it is when `executeNextTx` runs, not as it was when the transaction was queued (`Delay.sol:208–211`). So the veto window lasts `txCooldown` only while `txCooldown` is held constant, as Proposition 2 holds it. `txExpiration` only limits how long after the cooldown a transaction can still be executed (`Delay.sol:212–218`), so it does not affect the veto window. **FV — Once a veto moves `txNonce` past a transaction, that transaction can never be executed:** • DP-32 `vetoedTransactionCannotExecute` ✅ passes: a module queues a transaction, the owner's `setTxNonce` moves `txNonce` past it, and `executeNextTx` with that transaction then reverts. • DP-33 `entryPassedByTxNonceNeverExecutes` ✅ passes: from any state, once `txNonce` is past a queue entry, `executeNextTx` with that entry's transaction reverts, unless the same transaction has been queued again and sits at `txNonce`. Queuing it again makes a new entry, with its own veto window. • DP-27 `txNonceNeverDecreases` ✅ passes: no write function moves `txNonce` backwards, so a vetoed entry stays vetoed, whatever later happens to `txCooldown` or `txExpiration`. **On-chain — The veto window is 24 hours:** • ✅ At block 26114847, `txCooldown()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) is 86,400 seconds (24 hours). **FV — The TERM token holders can veto Delay Modifier transactions within the 24-hour veto window, through a proposal made by a holder with at least the proposal threshold of voting power that reaches the quorum, proposed to the Governor and executed once it passes the vote:** • GE-1 `termHoldersVetoThroughTheGovernor` ✅ passes. In one execution: ◦ A TERM token holder proposes the veto to the Governor: `setTxNonce(n)` on the Delay Modifier, for an `n` the Delay Modifier accepts. ◦ A TERM token holder votes for it. The proposal starts with no votes, so the Governor's own counting of TERM votes is what passes it. ◦ Once the vote has ended, anyone's `execute` succeeds with the proposal `Succeeded`. The call goes through the Roles Modifier and the DelayOwnerSafe to the Delay Modifier, whose owner is the DelayOwnerSafe (P2.3). ◦ The Delay Modifier's `txNonce` becomes `n`.

#	Statement	Evidence
		• GE-1's proposal meets both bars. Its `propose` succeeds, so by GP-1 (P2.12) the proposer has at least the proposal threshold of voting power. It is `Succeeded`, so by GS-17 its For and Abstain votes reach the quorum. • GS-17 `succeededProposalReachedQuorum` ✅ passes: a proposal is `Succeeded` only once its For and Abstain votes together reach the quorum at its snapshot. • GS-18 `proposalCanSucceedWithAPositiveQuorum` ✅ passes: a proposal can be `Succeeded` with a quorum above zero to reach. So GS-17 is not vacuous. • In GE-1, the DelayOwnerSafe is a stand-in that forwards the Roles Modifier's call. GV-3 `delayOwnerSafeLandsTheVeto` ✅ passes: on the real GnosisSafe v1.4.1, an enabled module's `execTransactionFromModule(Delay Modifier, 0, setTxNonce(n), Call)` succeeds and sets the Delay Modifier's `txNonce` to `n`. The Roles Modifier is the DelayOwnerSafe's module (P2.7), so the real DelayOwnerSafe lands the veto the same way. • The veto lands within the 24-hour veto window. The Governor's voting delay is 0 and its voting period is 79,200 seconds (22 hours), on a timestamp clock. GS-2 `governorSettingsNeverChange` and GS-3 `governorSettingsAreTheDeployedValues` ✅ pass: no function changes the voting delay or voting period, and the Governor's code fixes them at those values. A proposal made at time `p` can be executed from `p + 79,201`, once its deadline has passed, and the veto window closes 86,400 seconds after the transaction is queued. So a veto proposed within 7,198 seconds (just under 2 hours) of the transaction being queued can pass and execute within the veto window. • DP-35 `ownerCanSetTxNonceAtAnyTime` ✅ passes: whatever the time, `txCooldown`, `txExpiration` and pause state, the Delay Modifier owner's `setTxNonce(n)` succeeds and sets `txNonce` to `n`, for any `n` the Delay Modifier accepts. So the veto lands whenever the Governor's `execute` runs. • GE-1 assumes the Roles Modifier's target is the DelayOwnerSafe and its guard is the SetTxNonceGuard (read below). **On-chain — The setup formally verified above is deployed:** • ✅ At block 26114847, `target()` on the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) returns the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) and `guard()` returns the SetTxNonceGuard (`0x7aE03372ECDcEe335CdEfB7d354d507F0506616C`). • ✅ At block 26114847, on the Governor (`0x2B715634134220ffeEE9458b4e34E41A41418607`) `votingDelay()` is 0, `votingPeriod()` is 79,200 and `CLOCK_MODE()` is `mode=timestamp&from=default`. **On-chain — The proposal threshold is 1,000 TERM of voting power and the quorum is 1,000,000 TERM:** • ✅ At block 26237404, on the Governor (`0x2B715634134220ffeEE9458b4e34E41A41418607`) `proposalThreshold()` is 1,000 TERM (1e21), and `quorum(t)` for `t` one second before the block is 1,000,000 TERM (1e24). That is 1% of the TERM token's `totalSupply()` of 100,000,000 TERM (1e26) at that block. **FV — The Term multisig holders on the DelayOwnerSafe can veto Delay Modifier transactions within the 24-hour veto window, directly from the DelayOwnerSafe:** • DV-1 `approvedOwnersVetoThroughExecTransaction` ✅ passes. In one execution: ◦ An owner of the DelayOwnerSafe (a Term multisig holder, P2.6) approves a `setTxNonce(n)` transaction with `approveHash`, for an `n` the Delay Modifier accepts. ◦ `execTransaction` runs it with that approval as the signature. ◦ With the DelayOwnerSafe as the Delay Modifier's owner (P2.3), the call succeeds and sets the Delay Modifier's `txNonce` to `n`. • DV-1 sets the DelayOwnerSafe's threshold to 1. By SE141-6 (P2.4), with the real threshold of 5 (P2.6), five owners' approvals take the same path. • DV-1 assumes the DelayOwnerSafe owns the Delay Modifier (P2.3) and has no guard. The DelayOwnerSafe's guard is ConfigLockGuard (read below), which the next two bullets cover. The holders can veto at any point in the 24-hour veto window: by DP-35 (above), the owner's `setTxNonce` succeeds whatever the time. • ConfigLockGuard lets the veto through. CL141-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedModifier` is, the guard accepts every plain call that is not a locked call. The veto, `setTxNonce(n)` on the Delay Modifier, is a plain call. It is not one of the seven calls the guard locks on its locked modifier (CL141-6 to CL141-9). The guard's `checkAfterExecution` does nothing

#	Statement	Evidence
		(`ConfigLockGuard.sol:144`), so DV-1's path is unchanged. • OD-4 `guardedSafeCanStillVeto` ✅ passes. In this rule, ConfigLockGuard is installed on the DelayOwnerSafe with the Delay Modifier as its locked modifier, and the DelayOwnerSafe owns the Delay Modifier. An `execTransaction` to the Delay Modifier of `setTxNonce(n)` succeeds and `txNonce` becomes `n`, for an `n` the Delay Modifier accepts. • OD-4 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE141-5, SE141-12), and their approvals pass it (SE141-6). **On-chain — The setup formally verified above is deployed:** • ✅ At block 26135270, the DelayOwnerSafe's guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:42` in GnosisSafe v1.4.1) holds ConfigLockGuard (`0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53`). This is the same read as P5.DelayOwnerSafe.10. • ✅ At block 26135270, `lockedModifier()` on ConfigLockGuard returns the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`).

Proposition 3 — Only the Term multisig holders on the PauseSafe may pause, and only the Term multisig holders on the AdminSafe may unpause, the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.

#	Statement	Evidence
P3.1	Only the PauseGuard pauser and the PauseGuard ADMIN_ROLE holder can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state, and except `grantRole`, `revokeRole` and `renounceRole`, which revert for every caller.	**FV — Callers can only change the PauseGuard through `pause`, `unpause` and `setPauser`, and `grantRole`, `revokeRole` and `renounceRole` always revert:** • DP-9 `pausedOnlyChangesThroughPauseOrUnpause` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, `paused` changes only through `pause` or `unpause`. • DP-14 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-12 `renounceRoleAlwaysReverts` ✅ passes: no holder can drop a role. • DP-13 `inheritedGrantAndRevokeAlwaysRevert` ✅ passes: `grantRole` and `revokeRole` revert for every role and every caller, admins included, because both roles are administered by `DEFAULT_ADMIN_ROLE`, which nobody holds. • `checkTransaction` and `checkAfterExecution` are `view` functions (`PauseGuard.sol:114`, `PauseGuard.sol:120`), so they change no state. **FV — `pause` is only callable by the PauseGuard pauser, and `unpause` and `setPauser` only by the PauseGuard ADMIN_ROLE holder:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-6 `unpauseRevertsWithoutAdminRole` ✅ passes: without ADMIN_ROLE, `unpause` reverts, whoever the caller is. • DP-10 `setPauserRevertsWithoutAdminRole` ✅ passes: without ADMIN_ROLE, `setPauser` reverts.

Pause

P3.Pause: Only the Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Pause.1	Pause Delay Modifier executions (`PauseGuard.pause()`) is only callable by the PauseGuard pauser, and not by the PauseGuard ADMIN_ROLE holder.	**FV — `pause` is only callable by the PauseGuard pauser, and not by the PauseGuard ADMIN_ROLE holder:** • DP-3 `pauseRevertsForAnyoneButThePauser` ✅ passes: anyone who is not the pauser is rejected. • DP-4 `adminAloneCannotPause` ✅ passes: holding ADMIN_ROLE does not let a caller pause. • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. So DP-3 and DP-4 do not hold just because `pause` always reverts.
P3.Pause.2	`PauseGuard pauser = PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — The PauseGuard's pauser is the PauseSafe:** • ✅ At block 26229969, `pauser()` on the PauseGuard (`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`) returns the PauseSafe (`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`).
P3.Pause.3	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4. The PauseSafe runs GnosisSafe v1.4.1, like the DelayOwnerSafe (P3.Pause.5).
P3.Pause.4	PauseSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — PauseSafe fallbackHandler is `address(0)`:** • ✅ The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:14` in GnosisSafe v1.4.1) on the PauseSafe (`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`) is `address(0)` at block 26229969.
P3.Pause.5	`Owners of PauseSafe = Term multisig holders on PauseSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — Owners of PauseSafe are these nine Term multisig holders, and two of them must approve each transaction:** ✅ At block 26229969, the PauseSafe (`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`) runs GnosisSafe v1.4.1 (`VERSION`), has threshold 2 (`getThreshold`) and these 9 owners (`getOwners`), each an externally owned account with no contract code: • `0xeee661edcFE634Dc0e29D62C26AfD62c0843b817` • `0xF6A745f9B38FFcd17Ee8909AC89151D788F95282` • `0xbfFcAdCd5549cC378693108BcD4435776A6fa795` • `0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3` • `0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60` • `0xc1047a4D9f6071B55585523F7F659F39A1fBA74b` • `0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81` • `0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D` • `0xC3CbFc5DA4B3d4B1258D63cA7ba56518C33f28c7`
P3.Pause.6	`Modules of PauseSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard,	**On-chain — The Modules of PauseSafe list is empty:** • ✅ `getModulesPaginated` on the PauseSafe (`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`) returns an empty list at block 26229969.

#	Statement	Evidence
	txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	
P3.Pause.7	The Term multisig holders on the PauseSafe may pause the execution of Delay Modifier transactions, and if a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed.	**FV — The PauseSafe's owners can have it pause the PauseGuard, and a pause stops every Delay Modifier execution:** • DP-5 `pauseSucceedsForThePauser` ✅ passes: from an unpaused state, the pauser's `pause()` succeeds and `paused` is then true. • SE141-6 `ownersCanMakeTheSafeAct` ✅ passes: once the signature threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the PauseSafe's owners (P3.Pause.5) can have it call `pause` on the PauseGuard. • DP-18 `pausedBlocksExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, `executeNextTx` reverts and nothing reaches the Delay Modifier's target, for any transaction arguments and any queue state. • DP-19 `checkTransactionRevertsWhilePaused` ✅ passes: called directly, a paused PauseGuard rejects every transaction from every caller. • DP-34 `onlyExecuteNextTxReachesTheTarget` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, only `executeNextTx` hands a transaction to the Delay Modifier's target. With DP-18, a pause stops every transaction the Delay Modifier can send. **FV — If a pause outlasts the 24-hour expiration period following the cooldown, the Delay Modifier transaction can only be skipped or vetoed:** • DP-18 `pausedBlocksExecuteNextTx` ✅ passes (above): while the PauseGuard is paused, `executeNextTx` reverts, so the transaction cannot be executed during the pause. • DP-31 `executeNextTxRevertsAfterExpiration` ✅ passes: with a non-zero `txExpiration`, once more than `txCooldown` plus `txExpiration` has passed since the head entry was queued, `executeNextTx` with it reverts. That stays true at every later time, so a transaction whose window ended during the pause cannot be executed after the unpause either. • DM-12 `anyoneCanSkipExpired` ✅ passes: whether a call to `executeNextTx` or `skipExpired` succeeds, and what it changes, does not depend on who the caller is. DM-18 `outsiderCanSkipExpired` ✅ passes: a caller that is neither the owner nor a module can skip a queue entry. `skipExpired` (`Delay.sol:227`) moves `txNonce` past every expired entry at the head of the queue, so anyone can skip it. • The Owner of Delay Modifier can also move `txNonce` past it with a veto, `setTxNonce` (P2.2). • DP-33 `entryPassedByTxNonceNeverExecutes` ✅ passes: once `txNonce` is past the entry, `executeNextTx` with that entry's transaction reverts, unless the same transaction has been queued again and sits at `txNonce`. Queuing it again makes a new entry, with its own veto window. **On-chain — The PauseGuard is the Delay Modifier's guard, and it is not paused:** • ✅ At block 26135270, `guard()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) returns the PauseGuard (`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`), read in P1.13. • ✅ At block 26229969, `paused()` on the PauseGuard returns false. **On-chain — The Delay Modifier's expiration period is 24 hours:** • ✅ At block 26229805, `txExpiration()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) is 86,400 seconds (24 hours), read in P1.13.

Admin

P3.Admin: Only the Term multisig holders on the AdminSafe may unpause the execution of Delay Modifier transactions.

#	Statement	Evidence
P3.Admin.1	Unpause Delay Modifier executions (`PauseGuard.unpause()`) is only callable by the PauseGuard ADMIN_ROLE holder, and not by the PauseGuard pauser.	**FV — `unpause` is only callable by the PauseGuard ADMIN_ROLE holder, and not by the PauseGuard pauser:** • DP-6 `unpauseRevertsWithoutAdminRole` ✅ passes: without ADMIN_ROLE, `unpause` reverts, whoever the caller is. • DP-7 `pauserAloneCannotUnpause` ✅ passes: being the pauser, without ADMIN_ROLE, does not let a caller lift a pause. • DP-8 `unpauseSucceedsForAdminRole` ✅ passes: from a paused state, an ADMIN_ROLE holder's `unpause()` succeeds and `paused` is then false. So DP-6 and DP-7 do not hold just because `unpause` always reverts.
P3.Admin.2	`PauseGuard ADMIN_ROLE holder = AdminSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — The AdminSafe holds ADMIN_ROLE on the PauseGuard:** • ✅ At block 26229969, `hasRole(ADMIN_ROLE, 0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774)` on the PauseGuard (`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`) returns true for the AdminSafe, with `ADMIN_ROLE()` = `0xa49807205ce4d355092ef5a8a18f56e8913cf4a201fbe287825b095693c21775`. **FV — ADMIN_ROLE has exactly one holder:** • PD-3 `exactlyOneAdminRoleHolder` ✅ passes: from the constructor on, ADMIN_ROLE has exactly one holder. So the AdminSafe is the only ADMIN_ROLE holder.
P3.Admin.3	Only Modules of AdminSafe and Owners of AdminSafe can interact with the AdminSafe, except `execTransaction`, which anyone can call only if Owners of AdminSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2. The AdminSafe runs GnosisSafe v1.3.0, like the OwnerlessSafe (P3.Admin.5).
P3.Admin.4	AdminSafe fallbackHandler is `address(0)`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — AdminSafe fallbackHandler is `address(0)`:** • ✅ The fallbackHandler slot (`0x6c9a6c4a39284e37ed1cf53d337577d14212a4870fb976a4366c693b939918d5`, `FallbackManager.sol:12` in GnosisSafe v1.3.0) on the AdminSafe (`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`) is `address(0)` at block 26229969.
P3.Admin.5	`Owners of AdminSafe = Term multisig holders on AdminSafe`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — Owners of AdminSafe are these ten Term multisig holders, and four of them must approve each transaction:** ✅ At block 26229969, the AdminSafe (`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`) runs GnosisSafe v1.3.0 (`VERSION`), has threshold 4 (`getThreshold`) and these 10 owners (`getOwners`), each an externally owned account with no contract code: • `0xF6A745f9B38FFcd17Ee8909AC89151D788F95282` • `0x8EF485fD38b7B29a827938C9F00300eaBf8E1710` • `0xbfFcAdCd5549cC378693108BcD4435776A6fa795` • `0xB680373c50E9E877DA7dCF9efcc0dFB234452ad3` • `0x6eb0c274EC4B1d51152e45BAED22f883B2A3Bc60` • `0xE82183cfAE1C24f044315c318FD97bE7e47b31D2` • `0xeee661edcFE634Dc0e29D62C26AfD62c0843b817` • `0xc1047a4D9f6071B55585523F7F659F39A1fBA74b` • `0xDB97c377F23a9Dc54c3D6F9df7bC0ca4Cf7D5A81` • `0x89562FC5AEF155481aE0C7dde1300110B4dF2F1D`

#	Statement	Evidence
P3.Admin.6	`Modules of AdminSafe = null`, holding: 1. the owners, fallbackHandler, modules, guard constant for the PauseSafe and AdminSafe, and 2. the guard, txCooldown, txExpiration constant for the Delay Modifier, and 3. the pauser and ADMIN_ROLE holder constant for the PauseGuard.	**On-chain — The Modules of AdminSafe list is empty:** • ✅ `getModulesPaginated` on the AdminSafe (`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`) returns an empty list at block 26229969.
P3.Admin.7	The Term multisig holders on the AdminSafe may unpause the execution of Delay Modifier transactions.	**FV — The AdminSafe's owners can have it unpause the PauseGuard, and unpausing reopens Delay Modifier execution:** • DP-8 `unpauseSucceedsForAdminRole` ✅ passes: from a paused state, an ADMIN_ROLE holder's `unpause()` succeeds and `paused` is then false. • SE-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the AdminSafe's owners (P3.Admin.5) can have it call `unpause` on the PauseGuard. • DP-20 `unpauseReopensExecuteNextTx` ✅ passes: with the PauseGuard installed and paused, after an ADMIN_ROLE holder unpauses, a following `executeNextTx` can succeed and reach the Delay Modifier's target. • DP-21 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction from any caller, so it adds no revert of its own to `executeNextTx`. **On-chain — The PauseGuard is the Delay Modifier's guard:** • ✅ At block 26135270, `guard()` on the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`) returns the PauseGuard (`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`), read in P1.13.

Proposition 4 — The following cannot be changed by anyone:

1. the owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier
2. the voting token, proposal threshold and voting threshold for the Governor

The quorum for the Governor is 1% of TERM's total supply when a proposal is made; TERM's total supply can be changed only by the Term multisig holders on the ProposerSafe and OwnerlessSafe.

Roles Modifier

P4.RolesModifier: The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier cannot be changed by anyone.

#	Statement	Evidence
P4.RolesModifier.1	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P4.RolesModifier.2	The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier can only be changed by the Roles Modifier Owner, and not by Roles Modifier Modules with AssignedRole.	**FV — The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier can only be changed by the Roles Modifier Owner, not by Roles Modifier Modules with AssignedRole:** • RI-2 `rolesConfigOnlyChangesThroughOwner` ✅ passes: over every write function except `setUp`, the owner, target, module list, guard, role membership, default roles and target clearances change only when the caller is the Roles Modifier Owner. • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: `setUp`, the one write function RI-2 skips, always reverts, so it cannot reset these settings.

#	Statement	Evidence
		• RI-5 `onlyOwnerCanCallRolesSettings` ✅ passes: RI-2 does not track function and parameter scoping, so RI-5 covers it by call: `transferOwnership`, `renounceOwnership`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles` and `setDefaultRole` succeed only when the caller is the Roles Modifier Owner. • By P2.9, the Roles Modifier Owner is not a module, so no Roles Modifier Module with AssignedRole can change these settings.
P4.RolesModifier.3	`Roles Modifier Owner = address(0)`, which cannot be changed by anyone.	**On-chain — Roles Modifier Owner is `address(0)`:** • ✅ At block 26101471, `owner()` on the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) returns `address(0)`, and the `_owner` slot (slot 51, `OwnableUpgradeable`) holds zero. The OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) renounced ownership at block 26101459 (`OwnershipTransferred` to `address(0)`, tx `0xccd4324dd062785a3a4c8247268b89f5d10ad73b318abf7bcb7f4486c7add46a`), through a proposal run by the Delay Modifier's `executeNextTx`. **FV — Roles Modifier Owner cannot change from `address(0)`:** • RI-16 `noOwnerStaysNoOwner` ✅ passes: with the owner (Roles Modifier Owner) at `address(0)`, no function other than `setUp` can give the Roles Modifier an owner again, including `transferOwnership`. • RI-1 `setUpAlwaysRevertsAfterDeployment` ✅ passes: `setUp`, the one function RI-16 skips, always reverts, so it cannot set a new owner.
P4.RolesModifier.4	The owner, target, modules, guard, role targets, role scopes and role assignments for the Roles Modifier cannot be changed by anyone because `Roles Modifier Owner = address(0)`.	**FV — With the owner at `address(0)`, none of these settings can be changed by anyone:** • By P4.RolesModifier.2, only the Roles Modifier Owner can change these settings, and by P4.RolesModifier.3, the owner is `address(0)` for good. • RI-15 `noOwnerSettingsAlwaysRevert` ✅ passes: with the owner (Roles Modifier Owner) at `address(0)`, `transferOwnership`, `renounceOwnership`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `allowTarget`, `revokeTarget`, `scopeTarget`, `scopeAllowFunction`, `scopeRevokeFunction`, `scopeFunction`, `scopeFunctionExecutionOptions`, `scopeParameter`, `scopeParameterAsOneOf`, `unscopeParameter`, `assignRoles` and `setDefaultRole` revert for every caller except `address(0)`, which never sends a transaction.

Governor

P4.Governor: The voting token, proposal threshold and voting threshold for the Governor cannot be changed by anyone.

#	Statement	Evidence
P4.Governor.1	Only Holders of Governor Vote Token can interact with the Governor, except `execute`, which anyone can call only if Holders of Governor Vote Token proposed and passed the proposal, and except the `castVote` functions (`castVote`, `castVoteWithReason`, `castVoteWithReasonAndParams`, `castVoteBySig`, `castVoteWithReasonAndParamsBySig`), which anyone can call but which count a non-holder's vote as 0, and except `receive`, `onERC721Received`, `onERC1155Received` and `onERC1155BatchReceived`,	Same as P2.12.

#	Statement	Evidence
	which anyone can call but which make no storage writes.	
P4.Governor.2	The voting token, proposal threshold and voting threshold for the Governor cannot be changed by Holders of Governor Vote Token, so they cannot be changed by anyone.	**FV — The voting token, proposal threshold and voting threshold for the Governor cannot be changed by Holders of Governor Vote Token, so they cannot be changed by anyone:** • GS-1 `governorWriteFunctionsAreTheKnownFourteen` ✅ passes: the Governor's write functions are exactly the fourteen listed in P2.12. • GS-2 `governorSettingsNeverChange` ✅ passes: no write function changes the voting token or proposal threshold, whoever calls it and with whatever arguments. This includes `relay` called by the Governor as an action of a passed proposal. • GS-4 `queueAlwaysReverts` ✅ passes: `queue`, the one write function GS-2 skips, never succeeds. So none of the fourteen changes the voting token or proposal threshold, whoever calls it. • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: the proposal threshold (1,000 TERM) is fixed in the Governor's code, and the voting token is an immutable, so nothing the Governor stores can change them. • GS-16 `votingThresholdIsTheDeployedRule` ✅ passes: the voting threshold is fixed in the Governor's code (`GovernorCountingSimple.sol:67`). In every state, a proposal passes the vote exactly when its For votes are strictly more than its Against votes. The only stored data it reads is the tallies, and only votes move them (GC-1 to GC-4, P2.12).

TERM token

P4.TermToken: The quorum for the Governor is 1% of TERM's total supply when a proposal is made; TERM's total supply can be changed only by the Term multisig holders on the ProposerSafe and OwnerlessSafe.

#	Statement	Evidence
P4.TermToken.1	The quorum for the Governor is 1% of TERM's total supply when a proposal is made.	**FV — The quorum for the Governor is 1% of TERM's total supply when a proposal is made:** • GS-3 `governorSettingsAreTheDeployedValues` ✅ passes: at any past timepoint `t`, `quorum(t)` is TERM's `getPastTotalSupply(t)` divided by 100, and the voting delay is 0. • A proposal's snapshot is `clock() + votingDelay()` when it is made (`Governor.sol:329`), so with a voting delay of 0 it is the moment the proposal is made. • GS-17 `succeededProposalReachedQuorum` ✅ passes: a proposal is `Succeeded` only once its For and Abstain votes together reach the quorum at its snapshot. So the Governor checks a proposal's quorum at its snapshot. • GS-2 `governorSettingsNeverChange` ✅ passes: no write function changes the quorum numerator (1), the quorum denominator (100) or the voting delay. • By P2.13, the Governor's vote token is the TERM token. **FV — Once the second a proposal is made in has passed, TERM's total supply for that second never changes, except through `upgradeToAndCall`, so neither does the proposal's quorum:** • TT-4 `supplyCheckpointsOnlyChangeAtTheCurrentTimepoint` ✅ passes: over every write function except `upgradeToAndCall`, a total-supply checkpoint keyed before the current timestamp keeps its key and value, and the checkpoints are never shortened. • TT-5 `supplyCheckpointsGrowOnlyAtTheCurrentTimepoint` ✅ passes: over every write function except `upgradeToAndCall`, at most one total-supply checkpoint is appended, and it is keyed at the current timestamp. • TT-6 `supplyCheckpointsCanGrowAtTheCurrentTimepoint` ✅ passes: with minting unpaused, a DEFAULT_ADMIN_ROLE holder's `mint` can append a checkpoint. So TT-5 is not vacuous. • `getPastTotalSupply(t)` only answers for a `t` before the current timestamp (`VotesUpgradeable.sol:133`), and returns the value of the latest checkpoint keyed at or before `t` (`upperLookupRecent`, `Checkpoints.sol:265`). The checkpoints stay in key order, since a write keyed before the latest reverts (`Checkpoints.sol:335`). By TT-4 and TT-5, no write function except `upgradeToAndCall` changes or adds a

#	Statement	Evidence
		checkpoint keyed at or before such a `t`, so `getPastTotalSupply(t)` never changes. • A mint or burn later in the same second as the proposal still counts, because that second's checkpoint can change until the second has passed. So the quorum is 1% of TERM's total supply at the end of the second the proposal is made in. • Only the UPGRADER_ROLE holder can call `upgradeToAndCall` (P4.TermToken.2), and TERM runs the code the TT rules check (P4.TermToken.2).
P4.TermToken.2	TERM's total supply can be changed only through `mint` and `burn`, which only the DEFAULT_ADMIN_ROLE holder can call and which always revert while minting or burning, respectively, is paused, and through `upgradeToAndCall`, which only the UPGRADER_ROLE holder can call. Only the PAUSER_ROLE holder can pause or unpause minting and burning.	**FV — Callers can only interact with TERM through its nineteen write functions:** • TT-1 `termTokenWriteFunctionsAreTheKnownNineteen` ✅ passes: TermToken has no fallback, no receive and no write functions besides `transfer`, `transferFrom`, `approve`, `permit`, `delegate`, `delegateBySig`, `mint`, `burn`, `pause`, `unpause`, `pauseMinting`, `unpauseMinting`, `pauseBurning`, `unpauseBurning`, `grantRole`, `revokeRole`, `renounceRole`, `upgradeToAndCall` and `initialize`. TT-2 and TT-15 below run over every one of them. **FV — TERM's total supply changes only through `mint`, `burn` and `upgradeToAndCall`:** • TT-2 `totalSupplyOnlyChangesThroughMintOrBurn` ✅ passes: over every write function except `upgradeToAndCall` and `initialize`, TERM's total supply changes only through `mint` or `burn`. • TT-7 `initializeAlwaysRevertsOnceInitialized` ✅ passes: once TERM is initialized, `initialize`, which also mints, always reverts. TERM is initialized (read below). **FV — `mint` and `burn` are only callable by the DEFAULT_ADMIN_ROLE holder and always revert while minting or burning, respectively, is paused, and `upgradeToAndCall` is only callable by the UPGRADER_ROLE holder:** • TT-8 `onlyTheDefaultAdminCanMintOrBurn` ✅ passes: a successful `mint` or `burn` came from a DEFAULT_ADMIN_ROLE holder. • TT-13 `onlyTheUpgraderCanUpgrade` ✅ passes: a successful `upgradeToAndCall` came from an UPGRADER_ROLE holder. • TT-9 `mintRevertsWhileMintingPaused`, TT-10 `burnRevertsWhileBurningPaused` ✅ pass: while `mintingNotPaused` is false, `mint` always reverts, and while `burningNotPaused` is false, `burn` always reverts, whoever calls them. • TT-11 `defaultAdminCanMint`, TT-12 `defaultAdminCanBurn`, TT-14 `upgraderCanUpgrade` ✅ pass: with its flag on, a DEFAULT_ADMIN_ROLE holder's `mint` can raise and `burn` can lower TERM's total supply, and an UPGRADER_ROLE holder's `upgradeToAndCall` can succeed. So TT-8 to TT-10 and TT-13 are not vacuous. **FV — Only the PAUSER_ROLE holder can pause or unpause minting and burning:** • TT-15 `mintingAndBurningPausesOnlyChangeThroughThePausingFunctions` ✅ passes: over every write function except `upgradeToAndCall`, `mintingNotPaused` and `burningNotPaused` change only through `pauseMinting`, `unpauseMinting`, `pauseBurning` and `unpauseBurning`. • TT-16 `onlyThePauserCanPauseOrUnpauseMintingAndBurning` ✅ passes: those four succeed only for a PAUSER_ROLE holder. • TT-17 `pauserCanPauseAndUnpauseMintingAndBurning` ✅ passes: a PAUSER_ROLE holder's call to each of the four can succeed and flip its flag. So TT-15 and TT-16 are not vacuous. **On-chain — TERM runs the TermToken the TT rules check, and is initialized:** • ⏳ pending migration. TERM's (`0xC3d21f79C3120A4fFda7A535f8005a7c297799bF`) ERC-1967 implementation slot (`0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc`) must hold the upgraded implementation, whose verified source is `src/TermToken.sol` at term-token commit `74b19ca41aebf28c5b7801e94f082aad867b0f17`, the code the TT rules check. At block 26243952 it still holds the pre-upgrade implementation `0xeC222d8AfB8b4E78C418ebc1ab2cA181f19FbadC`. • ✅ At block 26243952, TERM's Initializable version (slot `0xf0c57e16840df040f15088dc2f81fe391c3923bec73e23a9662efc9c229c6a00`) is 1, so TT-7 applies. That TermToken has no reinitializer, so the upgrade leaves it at 1.
P4.TermToken.3	`DEFAULT_ADMIN_ROLE holder = OwnerlessSafe` and `UPGRADER_ROLE holder = OwnerlessSafe`, where only the Term multisig holders on the ProposerSafe and	**On-chain — The OwnerlessSafe is the only holder of DEFAULT_ADMIN_ROLE and the only holder of UPGRADER_ROLE on TERM, and the ProposerSafe is the only holder of PAUSER_ROLE:** • ✅ Through block 26243962, TERM's (`0xC3d21f79C3120A4fFda7A535f8005a7c297799bF`) full Blockscout log history has three

#	Statement	Evidence
	OwnerlessSafe can make the OwnerlessSafe act. `PAUSER_ROLE holder = ProposerSafe`, where only the Term multisig holders on the ProposerSafe can make the ProposerSafe act.	`RoleGranted` events, all in its initialization transaction (block 19877764, tx `0xb2e430fed6f849be8057b5fe8d533bb515a8dcaf74cb055d540182a7a5cf248b`): DEFAULT_ADMIN_ROLE and UPGRADER_ROLE to the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`), and PAUSER_ROLE to the AdminSafe (`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`). It has no `RoleRevoked` and no `RoleAdminChanged` event. So the OwnerlessSafe is the only holder of DEFAULT_ADMIN_ROLE and of UPGRADER_ROLE, and nobody else ever has held either. • ⏳ pending migration. The ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) must be the only holder of PAUSER_ROLE, so PAUSER_ROLE must be granted to it and revoked from the AdminSafe. At block 26243962, `hasRole(PAUSER_ROLE, …)` on TERM returns true for the AdminSafe and false for the ProposerSafe. Same as P1.2 to P1.12 for who can make the OwnerlessSafe act, and P1.9 to P1.12 for who can make the ProposerSafe act.

Proposition 5 — The following: 1. the owners for the ProposerSafe, OwnerlessSafe, and DelayOwnerSafe, and 2. the txCooldown, txExpiration for the Delay Modifier can be changed only by the Term multisig holders on the ProposerSafe, OwnerlessSafe, or DelayOwnerSafe. The fallbackHandler, modules, guard for the OwnerlessSafe can be changed only by the Term multisig holders on the ProposerSafe. The following: 1. the fallbackHandler, modules, guard for the ProposerSafe and DelayOwnerSafe, and 2. the owner, target, modules, guard for the Delay Modifier cannot be changed by anyone.

ProposerSafe

P5.ProposerSafe: The owners for the ProposerSafe can only be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by anyone.

#	Statement	Evidence
P5.ProposerSafe.1	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P5.ProposerSafe.2	ProposerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of ProposerSafe and Modules of ProposerSafe.	**On-chain — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **FV — ProposerSafe fallbackHandler can only be changed by Owners of ProposerSafe or Modules of ProposerSafe:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE-5, SE-12). So only Modules of ProposerSafe or Owners of ProposerSafe can change the fallbackHandler.

#	Statement	Evidence
P5.ProposerSafe.3	`Modules of ProposerSafe = null`, which can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`.	**On-chain — The Modules of ProposerSafe list is empty:** • Same as P1.12. **FV — Modules of ProposerSafe can only be changed by Owners of ProposerSafe because `Modules of ProposerSafe = null`:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of ProposerSafe as the threshold have approved (SE-5, SE-12). • The ProposerSafe has no module (P1.12), so only the Owners of ProposerSafe can change the module list.
P5.ProposerSafe.4	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the ProposerSafe because `Modules of ProposerSafe = null`.	**On-chain — Owners of ProposerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P1.11. **FV — Owners of ProposerSafe can only be changed by Term multisig holders on the ProposerSafe because `Modules of ProposerSafe = null`:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.9, only Modules of ProposerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE-5, SE-12). • The ProposerSafe has no module (P1.12), so only the Term multisig holders on the ProposerSafe can change the owners and threshold.
P5.ProposerSafe.5	The owners for the ProposerSafe can be changed by Term multisig holders on the ProposerSafe; the fallbackHandler, modules, guard for the ProposerSafe cannot be changed by Term multisig holders on the ProposerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the ProposerSafe can change Owners of ProposerSafe through `execTransaction`:** • CL-17 `guardedOwnersCanStillChangeOtherSettings` ✅ passes, for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL-17 stubs the signature check. By P1.9, `execTransaction` only acts once five Owners of ProposerSafe, the Term multisig holders (P1.11), have approved (SE-5, SE-12), and their approvals pass it (SE-6). **FV — The Term multisig holders on the ProposerSafe cannot change ProposerSafe fallbackHandler, Modules of ProposerSafe or ProposerSafe guard through `execTransaction` if ProposerSafe guard is ConfigLockGuard, so nobody can change them:** • CL-2 `checkTransactionRejectsSelfSetGuard`, CL-3 `checkTransactionRejectsSelfEnableModule`, CL-4 `checkTransactionRejectsSelfDisableModule`, CL-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL-8 `guardedSafeCannotCallSetGuard`, CL-9 `guardedSafeCannotCallEnableModule`, CL-10 `guardedSafeCannotCallDisableModule`, CL-11 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL-13 `guardedSelfCallNeverChangesGuard`, CL-14 `guardedSelfCallNeverChangesModules`, CL-15 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of ProposerSafe list and the fallbackHandler unchanged. • CL-1 `checkTransactionRejectsDelegateCall`, CL-12 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of ProposerSafe list or the fallbackHandler directly. • By P1.9, these settings only change when the caller is the Safe itself (SE-7), so an `execTransaction` to any other address cannot change them either. • These rules leave `lockedModifier` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.ProposerSafe.2 and P5.ProposerSafe.3, only Owners of ProposerSafe or Modules of ProposerSafe can change the fallbackHandler

#	Statement	Evidence
		and the Modules of ProposerSafe list, and SC-4 (P5.ProposerSafe.2) covers the guard the same way. The ProposerSafe has no module (P5.ProposerSafe.3), so nobody can change them. **On-chain — ProposerSafe guard is ConfigLockGuard:** • ✅ At block 26128252, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30`) on the ProposerSafe (`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`) holds ConfigLockGuard (`0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53`).

DelayOwnerSafe

P5.DelayOwnerSafe: The owners for the DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard for the DelayOwnerSafe cannot be changed by anyone.

#	Statement	Evidence
P5.DelayOwnerSafe.1	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.
P5.DelayOwnerSafe.2	DelayOwnerSafe fallbackHandler is `address(0)`, which can only be changed by Owners of DelayOwnerSafe and Modules of DelayOwnerSafe.	**On-chain — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **FV — DelayOwnerSafe fallbackHandler can only be changed by Owners of DelayOwnerSafe or Modules of DelayOwnerSafe:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once as many Owners of DelayOwnerSafe as the threshold have approved (SE141-5, SE141-12). So only Modules of DelayOwnerSafe or Owners of DelayOwnerSafe can change the fallbackHandler.

#	Statement	Evidence
P5.DelayOwnerSafe.3	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the DelayOwnerSafe and Modules of DelayOwnerSafe.	**On-chain — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **FV — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe or Modules of DelayOwnerSafe:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE141-5, SE141-12). So only Modules of DelayOwnerSafe or the Term multisig holders on the DelayOwnerSafe can change the owners and threshold.
P5.DelayOwnerSafe.4	`Modules of DelayOwnerSafe = Roles Modifier`, which can only be changed by Term multisig holders on the DelayOwnerSafe and Roles Modifier.	**On-chain — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • Same as P2.7. **FV — Modules of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe or Roles Modifier:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P2.4, only Modules of DelayOwnerSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE141-5, SE141-12). • The DelayOwnerSafe's only module is the Roles Modifier (P2.7), so only the Roles Modifier or the Term multisig holders on the DelayOwnerSafe can change the module list.
P5.DelayOwnerSafe.5	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P5.DelayOwnerSafe.6	`Roles Modifier Owner = address(0)`, which cannot be changed, P4.RolesModifier.	Same as P4.RolesModifier.3.
P5.DelayOwnerSafe.7	`Roles Modifier Modules with AssignedRole = Governor`, which cannot be changed by anyone, P4.RolesModifier.	Same as P2.10. See P4.RolesModifier: the Roles Modifier's modules and role assignments cannot be changed by anyone.
P5.DelayOwnerSafe.8	Governor is assigned only Role 1, which cannot be changed by anyone, P4.RolesModifier.	Same as P2.11. See P4.RolesModifier: the Governor's role assignments and default role cannot be changed by anyone.
P5.DelayOwnerSafe.9	Role 1 and the SetTxNonceGuard together prevent Governor from changing the owners, fallbackHandler, modules, guard for the DelayOwnerSafe. SetTxNonceGuard cannot be changed by anyone, P4.RolesModifier.	**FV — Role 1 and the SetTxNonceGuard together restrict the Governor to vetoes (`Delay.setTxNonce(uint256)`), so the Governor cannot change the owners, fallbackHandler, modules, guard for the DelayOwnerSafe:** • RG-2 `governorExecTransactionWithRoleLimitedToDelaySetTxNonce`, RG-3 `governorExecTransactionFromModuleLimitedToDelaySetTxNonce`, RG-4 `governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce`, RG-5 `governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce` ✅ pass: every call the Governor successfully executes through the Roles Modifier is `setTxNonce(uint256)` on the Delay Modifier, with zero value and as a plain `Call`, on all four execution entry points: `execTransactionWithRole`, `execTransactionFromModule`, `execTransactionWithRoleReturnData` and `execTransactionFromModuleReturnData`. • The two on-chain blobs below give the setup these rules assume: the SetTxNonceGuard installed and pointed at the Delay Modifier, and

#	Statement	Evidence
		the Governor an enabled module (P2.10), a member of Role 1 only, with default Role 1. • So through its module, the Roles Modifier, the Governor can only make the DelayOwnerSafe call the Delay Modifier's `setTxNonce`. The DelayOwnerSafe's settings only change when it calls itself (SE141-7, P2.4), so the Governor cannot change its owners, fallbackHandler, modules or guard. **On-chain — The SetTxNonceGuard is the Roles Modifier's guard and is pointed at the Delay Modifier:** • ✅ At block 26101511, `guard()` on the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) returns the SetTxNonceGuard (`0x7aE03372ECDcEe335CdEfB7d354d507F0506616C`), and `getGuard()` returns the same address. • ✅ At the same block, `delay()` on the SetTxNonceGuard returns the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`), and `target()` on the Roles Modifier returns the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`), not the Roles Modifier itself. **On-chain — Governor is assigned only Role 1:** • Same as P5.DelayOwnerSafe.8. **Already proven — SetTxNonceGuard cannot be changed by anyone:** • See P4.RolesModifier: the Roles Modifier's guard, the SetTxNonceGuard, cannot be changed by anyone. The guard's `delay` is `immutable` (`SetTxNonceGuard.sol:17`).
P5.DelayOwnerSafe.10	The owners for the DelayOwnerSafe can be changed by Term multisig holders on the DelayOwnerSafe; the fallbackHandler, modules, guard for the DelayOwnerSafe cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe can change Owners of DelayOwnerSafe through `execTransaction`:** • CL141-21 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on GnosisSafe v1.4.1, the DelayOwnerSafe's code (P2.4), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL141-21 stubs the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE141-5, SE141-12), and their approvals pass it (SE141-6). **FV — The Term multisig holders on the DelayOwnerSafe cannot change DelayOwnerSafe fallbackHandler, Modules of DelayOwnerSafe or DelayOwnerSafe guard through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL141-2 `checkTransactionRejectsSelfSetGuard`, CL141-3 `checkTransactionRejectsSelfEnableModule`, CL141-4 `checkTransactionRejectsSelfDisableModule`, CL141-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL141-12 `guardedSafeCannotCallSetGuard`, CL141-13 `guardedSafeCannotCallEnableModule`, CL141-14 `guardedSafeCannotCallDisableModule`, CL141-15 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL141-17 `guardedSelfCallNeverChangesGuard`, CL141-18 `guardedSelfCallNeverChangesModules`, CL141-19 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of DelayOwnerSafe list and the fallbackHandler unchanged. • CL141-1 `checkTransactionRejectsDelegateCall`, CL141-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of DelayOwnerSafe list or the fallbackHandler directly. • By P2.4, these settings only change when the caller is the Safe itself (SE141-7), so an `execTransaction` to any other address cannot change them either. • These rules run on GnosisSafe v1.4.1, the DelayOwnerSafe's code (P2.4), and leave `lockedModifier` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.DelayOwnerSafe.2 and P5.DelayOwnerSafe.4, only Owners of DelayOwnerSafe or Modules of DelayOwnerSafe can change the fallbackHandler and the Modules of DelayOwnerSafe list, and SC141-4 (P5.DelayOwnerSafe.2) covers the guard the same way. The only Module of DelayOwnerSafe is the Roles Modifier (P5.DelayOwnerSafe.4), and by P5.DelayOwnerSafe.5 to P5.DelayOwnerSafe.9 nobody

#	Statement	Evidence
		can change them through it, so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard:** • ✅ At block 26135270, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:42` in GnosisSafe v1.4.1) on the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`) holds ConfigLockGuard (`0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53`).

Delay Modifier

P5.DelayModifier: The txCooldown, txExpiration for the Delay Modifier can only be changed by Term multisig holders on the DelayOwnerSafe; the owner, target, modules, guard for the Delay Modifier cannot be changed by anyone.

#	Statement	Evidence
P5.DelayModifier.1	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P5.DelayModifier.2	The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not by Modules of Delay Modifier.	**FV — The owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier can only be changed by Owner of Delay Modifier, not Modules of Delay Modifier:** • DM-8 `atMostOneCallerPassesOnlyOwner` ✅ passes: each of the Delay Modifier's ten `onlyOwner` functions, which include `setTxCooldown`, `setTxExpiration`, `setTarget`, `enableModule`, `disableModule`, `setGuard`, `transferOwnership` and `renounceOwnership`, succeeds only when the caller is the owner (Owner of Delay Modifier). • DM-6 `ownerOnlyChangesThroughOwnableTransfer`, DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable`, DM-7 `guardOnlyChangesThroughOwnerSetGuard` ✅ pass: over every write function, the owner, the module list and the guard change only when the caller is the owner. • By P1.7, the owner is not a module, so no Module of Delay Modifier can change these settings.
P5.DelayModifier.3	`Owner of Delay Modifier = DelayOwnerSafe`, which can only be changed by the DelayOwnerSafe.	**On-chain — Owner is DelayOwnerSafe:** • ✅ Same as P2.3. **FV — Owner of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-6 `ownerOnlyChangesThroughOwnableTransfer` ✅ passes: over every write function, the owner changes only through `transferOwnership` or `renounceOwnership`, and only when the caller is the current owner (Owner of Delay Modifier). • So, since the owner is the DelayOwnerSafe, only the DelayOwnerSafe can change it.
P5.DelayModifier.4	Only Modules of DelayOwnerSafe and Owners of DelayOwnerSafe can interact with the DelayOwnerSafe, except `execTransaction`, which anyone can call only if Owners of DelayOwnerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P2.4.

#	Statement	Evidence
P5.DelayModifier.5	DelayOwnerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P5.DelayOwnerSafe.	**Already proven — DelayOwnerSafe fallbackHandler is `address(0)`:** • Same as P2.5. **Already proven — DelayOwnerSafe fallbackHandler cannot be changed by anyone:** • See P5.DelayOwnerSafe: by P5.DelayOwnerSafe.10, the Term multisig holders on the DelayOwnerSafe cannot change the fallbackHandler, so nobody can.
P5.DelayModifier.6	`Owners of DelayOwnerSafe = Term multisig holders on the DelayOwnerSafe`, which can only be changed by Term multisig holders on the DelayOwnerSafe, P5.DelayOwnerSafe.	**Already proven — Owners of DelayOwnerSafe are these eleven Term multisig holders, and five of them must approve each transaction:** • Same as P2.6. **Already proven — Owners of DelayOwnerSafe can only be changed by Term multisig holders on the DelayOwnerSafe:** • See P5.DelayOwnerSafe.
P5.DelayModifier.7	`Modules of DelayOwnerSafe = Roles Modifier`, which cannot be changed by anyone, P5.DelayOwnerSafe.	**Already proven — The Modules of DelayOwnerSafe list contains only the Roles Modifier:** • Same as P2.7. **Already proven — Modules of DelayOwnerSafe cannot be changed by anyone:** • See P5.DelayOwnerSafe: by P5.DelayOwnerSafe.10, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of DelayOwnerSafe list, so nobody can.
P5.DelayModifier.8	Only Roles Modifier Modules with AssignedRole or Roles Modifier Owner can interact with Roles Modifier.	Same as P2.8.
P5.DelayModifier.9	`Roles Modifier Owner = address(0)`, which cannot be changed, P4.RolesModifier.	Same as P4.RolesModifier.3.
P5.DelayModifier.10	`Roles Modifier Modules with AssignedRole = Governor`, which cannot be changed by anyone, P4.RolesModifier.	Same as P5.DelayOwnerSafe.7.
P5.DelayModifier.11	Governor is assigned only Role 1, which cannot be changed by anyone, P4.RolesModifier.	Same as P5.DelayOwnerSafe.8.
P5.DelayModifier.12	Role 1 and the SetTxNonceGuard together prevent Governor from changing the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier. SetTxNonceGuard cannot be changed by anyone, P4.RolesModifier.	**FV — Role 1 and the SetTxNonceGuard together restrict the Governor to vetoes (`Delay.setTxNonce(uint256)`), so the Governor cannot change the owner, target, modules, guard, txCooldown, txExpiration for the Delay Modifier:** • RG-2 `governorExecTransactionWithRoleLimitedToDelaySetTxNonce`, RG-3 `governorExecTransactionFromModuleLimitedToDelaySetTxNonce`, RG-4 `governorExecTransactionWithRoleReturnDataLimitedToDelaySetTxNonce`, RG-5 `governorExecTransactionFromModuleReturnDataLimitedToDelaySetTxNonce` ✅ pass: every call the Governor successfully executes through the Roles Modifier is `setTxNonce(uint256)` on the Delay Modifier, with zero value and as a plain `Call`, on all four execution entry points: `execTransactionWithRole`, `execTransactionFromModule`, `execTransactionWithRoleReturnData` and `execTransactionFromModuleReturnData`. • The two on-chain blobs below give the setup these rules assume: the SetTxNonceGuard installed and pointed at the Delay Modifier, and the Governor an enabled module (P2.10), a member of Role 1 only, with default Role 1. • So through the Roles Modifier, the Governor can only make the DelayOwnerSafe call the Delay Modifier's `setTxNonce`, which only writes `txNonce` (`Delay.sol:136–143`). So the Governor cannot change the Delay Modifier's owner, target, modules, guard, txCooldown or txExpiration.

#	Statement	Evidence
		On-chain — The SetTxNonceGuard is the Roles Modifier's guard and is pointed at the Delay Modifier: • ✅ At block 26101511, `guard()` on the Roles Modifier (`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`) returns the SetTxNonceGuard (`0x7aE03372ECDcEe335CdEfB7d354d507F0506616C`), and `getGuard()` returns the same address. • ✅ At the same block, `delay()` on the SetTxNonceGuard returns the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`), and `target()` on the Roles Modifier returns the DelayOwnerSafe (`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`), not the Roles Modifier itself. **On-chain — Governor is assigned only Role 1:** • Same as P5.DelayModifier.11. **Already proven — SetTxNonceGuard cannot be changed by anyone:** • See P4.RolesModifier: the Roles Modifier's guard, the SetTxNonceGuard, cannot be changed by anyone. The guard's `delay` is `immutable` (`SetTxNonceGuard.sol:17`).
P5.DelayModifier.13	The txCooldown, txExpiration for the Delay Modifier can be changed by Term multisig holders on the DelayOwnerSafe; the owner, target, modules, guard for the Delay Modifier cannot be changed by Term multisig holders on the DelayOwnerSafe, so they cannot be changed by anyone.	**FV — The Term multisig holders on the DelayOwnerSafe can change the txCooldown and txExpiration of the Delay Modifier through `execTransaction`:** • OD-5 `guardedSafeCanStillSetTxCooldown`, OD-6 `guardedSafeCanStillSetTxExpiration` ✅ pass: with ConfigLockGuard installed on the DelayOwnerSafe, an `execTransaction` to the Delay Modifier of `setTxCooldown(c)` succeeds and the txCooldown becomes `c`, and of `setTxExpiration(x)` succeeds and the txExpiration becomes `x`. • OD-5 and OD-6 assume the DelayOwnerSafe is the Delay Modifier's owner (P2.3). • OD-5 and OD-6 stub the signature check. By P2.4, `execTransaction` only acts once five Owners of DelayOwnerSafe, the Term multisig holders (P2.6), have approved (SE141-5, SE141-12), and their approvals pass it (SE141-6). **FV — The Term multisig holders on the DelayOwnerSafe cannot change the owner, target, modules or guard of the Delay Modifier through `execTransaction` if DelayOwnerSafe guard is ConfigLockGuard, so nobody can change them:** • CL141-6 `checkTransactionRejectsModifierOwnershipChange`, CL141-7 `checkTransactionRejectsModifierModuleChange`, CL141-8 `checkTransactionRejectsModifierSetGuard`, CL141-9 `checkTransactionRejectsModifierSetAvatarOrTarget` ✅ pass: called by the Safe, the guard reverts on any transaction to its locked modifier carrying `transferOwnership`, `renounceOwnership`, `enableModule`, `disableModule`, `setGuard`, `setAvatar` or `setTarget`. • CL141-23 `guardedSafeCannotCallLockedModifierFunctions` ✅ passes: with ConfigLockGuard installed, an `execTransaction` addressed to the locked modifier with any of these calls reverts, whatever the arguments, value and operation. • CL141-1 `checkTransactionRejectsDelegateCall`, CL141-16 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so every `execTransaction` to the Delay Modifier that succeeds is a plain call carrying some other selector. • OD-1 `unlockedDelayCallsNeverChangeOwnerGuardAvatarOrTarget`, OD-2 `unlockedDelayCallsNeverChangeModules` ✅ pass: over every Delay Modifier write function except those seven and `setUp`, and for any caller, the owner, guard, target and module list do not change. • An `execTransaction` to any other address reaches the Delay Modifier, if at all, with that address as the caller, not the owner, so it cannot change these settings (P5.DelayModifier.2). • The CL141 rules leave `lockedModifier` unconstrained, so they lock whichever modifier ConfigLockGuard was built with. The deployed guard's locked modifier is the Delay Modifier (below). • By P5.DelayModifier.2 and P5.DelayModifier.3, only the DelayOwnerSafe can change the owner, target, modules and guard. The DelayOwnerSafe acts only through `execTransaction` or its modules (P5.DelayModifier.4, P5.DelayModifier.5). Its only module is the Roles Modifier (P5.DelayModifier.7), through which nobody can change them (P5.DelayModifier.8 to P5.DelayModifier.12), so nobody can change them. **On-chain — DelayOwnerSafe guard is ConfigLockGuard, with the Delay Modifier as its locked modifier:** • ✅ Same as P5.DelayOwnerSafe.10: at block 26135270, the DelayOwnerSafe's guard slot holds ConfigLockGuard

#	Statement	Evidence
		(`0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53`). • ✅ At block 26128252, `lockedModifier()` on ConfigLockGuard returns the Delay Modifier (`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`).

OwnerlessSafe

P5.OwnerlessSafe: The owners for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.

#	Statement	Evidence
P5.OwnerlessSafe.1	Only Modules of OwnerlessSafe and Owners of OwnerlessSafe can interact with the OwnerlessSafe, except `execTransaction`, which anyone can call only if Owners of OwnerlessSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.2.
P5.OwnerlessSafe.2	OwnerlessSafe fallbackHandler is `address(0)`, which can only be changed by Modules of OwnerlessSafe and Owners of OwnerlessSafe.	**On-chain — OwnerlessSafe fallbackHandler is `address(0)`:** • Same as P1.3. **FV — OwnerlessSafe fallbackHandler can only be changed by Owners of OwnerlessSafe or Modules of OwnerlessSafe:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of OwnerlessSafe as the threshold have approved (SE-5, SE-12). So only Modules of OwnerlessSafe or Owners of OwnerlessSafe can change the fallbackHandler.
P5.OwnerlessSafe.3	`Owners of OwnerlessSafe = Term multisig holders on the OwnerlessSafe`, which can only be changed by Term multisig holders on the OwnerlessSafe and Modules of OwnerlessSafe.	**On-chain — Owners of OwnerlessSafe are these nine Term multisig holders, and all nine must approve each transaction:** • Same as P1.4. **FV — Owners of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Modules of OwnerlessSafe:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE-5, SE-12). So only Modules of OwnerlessSafe or the Term multisig holders on the OwnerlessSafe can change the owners and threshold.
P5.OwnerlessSafe.4	`Modules of OwnerlessSafe = Delay Modifier`, which can only be changed by Term multisig holders on the OwnerlessSafe and Delay Modifier.	**On-chain — The Modules of OwnerlessSafe list contains only the Delay Modifier:** • Same as P1.5. **FV — Modules of OwnerlessSafe can only be changed by Term multisig holders on the OwnerlessSafe or Delay Modifier:**

#	Statement	Evidence
		• SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P1.2, only Modules of OwnerlessSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE-5, SE-12). • The OwnerlessSafe's only module is the Delay Modifier (P1.5), so only the Delay Modifier or the Term multisig holders on the OwnerlessSafe can change the module list.
P5.Ownerless Safe.5	Only Modules of Delay Modifier or Owner of Delay Modifier can interact with Delay Modifier, except `executeNextTx` and `skipExpired`, which anyone can call, but they can only execute or skip transactions queued by Modules of Delay Modifier or Owner of Delay Modifier.	Same as P1.6.
P5.Ownerless Safe.6	Only Modules of Delay Modifier can queue transactions to Delay Modifier. Owner of Delay Modifier cannot queue transactions to Delay Modifier.	Same as P1.7.
P5.Ownerless Safe.7	`Modules of Delay Modifier = ProposerSafe`, which cannot be changed by anyone, P5.DelayModifier.	**On-chain — The Modules of Delay Modifier list contains only the ProposerSafe:** • Same as P1.8. **FV — Modules of Delay Modifier can only be changed by Owner of Delay Modifier:** • DM-2 `modulesOnlyChangeThroughOwnerEnableOrDisable` ✅ passes: over every write function, the module list changes only when the caller is the owner (Owner of Delay Modifier). **Already proven — Modules of Delay Modifier cannot be changed by anyone:** • See P5.DelayModifier: by P5.DelayModifier.13, the Term multisig holders on the DelayOwnerSafe cannot change the Modules of Delay Modifier, so nobody can.
P5.Ownerless Safe.8	Only Modules of ProposerSafe and Owners of ProposerSafe can interact with the ProposerSafe, except `execTransaction`, which anyone can call only if Owners of ProposerSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P1.9.
P5.Ownerless Safe.9	ProposerSafe fallbackHandler is `address(0)`, which cannot be changed by anyone, P5.ProposerSafe.	**Already proven — ProposerSafe fallbackHandler is `address(0)`:** • Same as P1.10. **Already proven — ProposerSafe fallbackHandler cannot be changed by anyone:** • See P5.ProposerSafe: by P5.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the fallbackHandler, so nobody can.
P5.Ownerless Safe.10	`Modules of ProposerSafe = null`, which cannot be changed by anyone, P5.ProposerSafe.	**Already proven — The Modules of ProposerSafe list is empty:** • Same as P1.12. **Already proven — Modules of ProposerSafe cannot be changed by anyone:**

#	Statement	Evidence
		• See P5.ProposerSafe: by P5.ProposerSafe.5, the Term multisig holders on the ProposerSafe cannot change the Modules of ProposerSafe list, so nobody can.
P5.Ownerless Safe.11	`Owners of ProposerSafe = Term multisig holders on the ProposerSafe`, which can only be changed by Term multisig holders on the ProposerSafe, P5.ProposerSafe.	Same as P5.ProposerSafe.4.
P5.Ownerless Safe.12	The owners for the OwnerlessSafe can be changed by Term multisig holders on the ProposerSafe and Term multisig holders on the OwnerlessSafe; the fallbackHandler, modules, guard for the OwnerlessSafe can only be changed by Term multisig holders on the ProposerSafe.	**FV — The Term multisig holders on the OwnerlessSafe can change Owners of OwnerlessSafe through `execTransaction`:** • CL-17 `guardedOwnersCanStillChangeOtherSettings` ✅ passes on GnosisSafe v1.3.0, the OwnerlessSafe's code (P1.2), for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner` and `changeThreshold`: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with a call to that function succeeds and the owners or threshold change. • CL-17 stubs the signature check. By P1.2, `execTransaction` only acts once all nine Owners of OwnerlessSafe, the Term multisig holders (P1.4), have approved (SE-5, SE-12), and their approvals pass it (SE-6). **FV — The Term multisig holders on the ProposerSafe can change the owners, fallbackHandler, modules and guard of the OwnerlessSafe through the Delay Modifier:** • PO-5 `proposerApprovalChangesOwnerlessSettings` ✅ passes, for each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler`, in one execution: ◦ An owner of the ProposerSafe (a Term multisig holder, P1.11) approves with `approveHash` the hash of the ProposerSafe's call to the Delay Modifier's `execTransactionFromModule(OwnerlessSafe, 0, <settings call>, Call)`, and anyone submits it through `execTransaction` with that approval as the signature. ◦ The ProposerSafe is the Delay Modifier's module (P1.8), so the Delay Modifier queues the proposal. ◦ Once the cooldown has passed and before the proposal expires, anyone's `executeNextTx` hands it to the Delay Modifier's target, the OwnerlessSafe, which has the Delay Modifier as its module (P1.5). ◦ The OwnerlessSafe's `execTransactionFromModule` makes it call its own settings function, and the setting changes. • PO-5 sets the ProposerSafe's threshold to 1. By SE-6 (P1.9), with the real threshold of 5 (P1.11), five owners' approvals take the same path. • PO-5 assumes no guard on either Safe or on the Delay Modifier. The Delay Modifier's guard is the PauseGuard. DP-21 `checkTransactionAcceptsWhileNotPaused` ✅ passes: while not paused, the PauseGuard accepts any transaction. Its `checkAfterExecution` does nothing (`PauseGuard.sol:120`), so with the PauseGuard not paused, PO-5's path is unchanged. • The ProposerSafe's guard is ConfigLockGuard (P5.ProposerSafe.5). The ProposerSafe's call is to the Delay Modifier's `execTransactionFromModule`, which is not one of the guard's locked calls. CL141-10 `checkTransactionAcceptsEverythingElse` ✅ passes: whatever `lockedModifier` is, the guard accepts every plain call that is not a locked call. It checks ConfigLockGuard's `checkTransaction` directly, so the Safe version does not matter. Its `checkAfterExecution` does nothing (`ConfigLockGuard.sol:144`), so PO-5's ProposerSafe step is unchanged. • The OwnerlessSafe's guard is ConfigLockGuard (below), but it is never consulted on this path: GnosisSafe v1.3.0 consults its guard only in `execTransaction` (`GnosisSafe.sol:147–191`), not in `execTransactionFromModule`. LG-2 `moduleCanStillChangeEachSetting` ✅ passes: with ConfigLockGuard installed and its `lockedModifier` the Delay Modifier (P5.DelayModifier.13), an `execTransactionFromModule` from the Delay Modifier, as an enabled module, addressed to the Safe with a call to each of `enableModule`, `disableModule`, `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `setGuard` and `setFallbackHandler` succeeds and a setting changes. • PO-5 assumes the deployed wiring: the ProposerSafe is a module of the Delay Modifier (P1.8), and the Delay Modifier is a module of the OwnerlessSafe (P1.5). The Delay Modifier's target is the OwnerlessSafe and its guard is the PauseGuard, read in P1.13. • While the PauseGuard is paused, it rejects every `executeNextTx` (`PauseGuard.sol:116`), so the proposal waits until a holder of the PauseGuard's `ADMIN_ROLE` unpauses it. If `txExpiration` passes first, the proposal expires and has to be proposed again. **FV — The Term multisig holders on the OwnerlessSafe cannot change OwnerlessSafe fallbackHandler, Modules of OwnerlessSafe**

#	Statement	Evidence
		or OwnerlessSafe guard through `execTransaction` if OwnerlessSafe guard is ConfigLockGuard, so only the Term multisig holders on the ProposerSafe can change them: • CL-2 `checkTransactionRejectsSelfSetGuard`, CL-3 `checkTransactionRejectsSelfEnableModule`, CL-4 `checkTransactionRejectsSelfDisableModule`, CL-5 `checkTransactionRejectsSelfSetFallbackHandler` ✅ pass: called by the Safe, the guard reverts on any transaction to the Safe itself carrying `setGuard`, `enableModule`, `disableModule` or `setFallbackHandler`. • CL-8 `guardedSafeCannotCallSetGuard`, CL-9 `guardedSafeCannotCallEnableModule`, CL-10 `guardedSafeCannotCallDisableModule`, CL-11 `guardedSafeCannotCallSetFallbackHandler` ✅ pass: with ConfigLockGuard installed, an `execTransaction` addressed to the Safe with any of these calls reverts. • CL-13 `guardedSelfCallNeverChangesGuard`, CL-14 `guardedSelfCallNeverChangesModules`, CL-15 `guardedSelfCallNeverChangesFallbackHandler` ✅ pass: for any calldata and operation, an `execTransaction` addressed to the Safe leaves the guard, every entry of the Modules of OwnerlessSafe list and the fallbackHandler unchanged. • CL-1 `checkTransactionRejectsDelegateCall`, CL-12 `guardedSafeCannotDelegateCall` ✅ pass: the guard rejects every delegate call, so no delegated code can write the guard, the Modules of OwnerlessSafe list or the fallbackHandler directly. • By P1.2, these settings only change when the caller is the Safe itself (SE-7), so an `execTransaction` to any other address cannot change them either. • These rules run on GnosisSafe v1.3.0, the OwnerlessSafe's code (P1.2), and leave `lockedModifier` unconstrained, so they hold whichever modifier ConfigLockGuard was built with. • By P5.OwnerlessSafe.2 and P5.OwnerlessSafe.4, only Owners of OwnerlessSafe or Modules of OwnerlessSafe can change the fallbackHandler and the Modules of OwnerlessSafe list, and SC-4 (P5.OwnerlessSafe.2) covers the guard the same way. The only Module of OwnerlessSafe is the Delay Modifier (P5.OwnerlessSafe.4), which only runs transactions queued by Modules of Delay Modifier (P5.OwnerlessSafe.5, P5.OwnerlessSafe.6). The only Module of Delay Modifier is the ProposerSafe, which cannot be changed by anyone (P5.OwnerlessSafe.7), and only the Term multisig holders on the ProposerSafe can make the ProposerSafe act (P5.OwnerlessSafe.8 to P5.OwnerlessSafe.11). So only the Term multisig holders on the ProposerSafe can change them. **On-chain — OwnerlessSafe guard is ConfigLockGuard:** • ✅ At block 26128252, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30`) on the OwnerlessSafe (`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`) holds ConfigLockGuard (`0xeda83f6d7A8A56D4AA5433Bf7cC1fef496036d53`).

Proposition 6 — The owners, fallbackHandler, modules, guard for the PauseSafe can be changed only by the Term multisig holders on the PauseSafe. The following: 1. the owners, fallbackHandler, modules, guard for the AdminSafe, and 2. the pauser for the PauseGuard can be changed only by the Term multisig holders on the AdminSafe. The ADMIN_ROLE holder for the PauseGuard cannot be changed by anyone.

PauseSafe

P6.PauseSafe: The owners, fallbackHandler, modules, guard for the PauseSafe can only be changed by Term multisig holders on the PauseSafe.

#	Statement	Evidence
P6.PauseSafe.1	Only Modules of PauseSafe and Owners of PauseSafe can interact with the PauseSafe, except `execTransaction`, which anyone can call only if Owners of PauseSafe have approved the transaction, and except	Same as P3.Pause.3.

#	Statement	Evidence
	`fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	
P6.PauseSafe.2	PauseSafe fallbackHandler is `address(0)`, which can only be changed by Owners of PauseSafe and Modules of PauseSafe.	Same as P3.Pause.4. **FV — PauseSafe fallbackHandler can only be changed by Owners of PauseSafe or Modules of PauseSafe:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE141-5, SE141-12). So only Modules of PauseSafe or Owners of PauseSafe can change the fallbackHandler.
P6.PauseSafe.3	`Modules of PauseSafe = null`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.6. **FV — Modules of PauseSafe can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE141-5, SE141-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the module list.
P6.PauseSafe.4	`Owners of PauseSafe = Term multisig holders on the PauseSafe`, which can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`.	Same as P3.Pause.5. **FV — Owners of PauseSafe can only be changed by Term multisig holders on the PauseSafe because `Modules of PauseSafe = null`:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once two Owners of PauseSafe, the Term multisig holders (P3.Pause.5), have approved (SE141-5, SE141-12). • The PauseSafe has no module (P3.Pause.6), so only the Term multisig holders on the PauseSafe can change the owners and threshold.
P6.PauseSafe.5	PauseSafe guard is `address(0)`, which can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`.	**On-chain — PauseSafe guard is `address(0)`:** • ✅ At block 26229969, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:42` in GnosisSafe v1.4.1) on the PauseSafe (`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`) is `address(0)`. **FV — PauseSafe guard can only be changed by Owners of PauseSafe because `Modules of PauseSafe = null`:** • SC141-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the guard only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.PauseSafe.1, only Modules of PauseSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE141-3), and `execTransaction` only acts once as many Owners of PauseSafe as the threshold have approved (SE141-5, SE141-12). • The PauseSafe has no module (P3.Pause.6), so only the Owners of PauseSafe can change the guard.

#	Statement	Evidence
P6.PauseSafe.6	The owners, fallbackHandler, modules, guard for the PauseSafe can be changed by Term multisig holders on the PauseSafe.	**FV — The Term multisig holders on the PauseSafe can change the owners, fallbackHandler, modules and guard for the PauseSafe through `execTransaction`:** • SC141-3 `ownersCanChangeSettings` ✅ passes: for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `enableModule`, `disableModule`, `setGuard` and `setFallbackHandler`, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. • SE141-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. The PauseSafe runs GnosisSafe v1.4.1 (P3.Pause.5), and its owners are the Term multisig holders on the PauseSafe (P3.Pause.5). • The PauseSafe has no guard (P6.PauseSafe.5), so nothing rejects the owners' `execTransaction`.

AdminSafe

P6.AdminSafe: The owners, fallbackHandler, modules, guard for the AdminSafe can only be changed by Term multisig holders on the AdminSafe.

#	Statement	Evidence
P6.AdminSafe.1	Only Modules of AdminSafe and Owners of AdminSafe can interact with the AdminSafe, except `execTransaction`, which anyone can call only if Owners of AdminSafe have approved the transaction, and except `fallback` and `receive`, which anyone can call but which do not change state, assuming the fallbackHandler is `address(0)`.	Same as P3.Admin.3.
P6.AdminSafe.2	AdminSafe fallbackHandler is `address(0)`, which can only be changed by Owners of AdminSafe and Modules of AdminSafe.	Same as P3.Admin.4. **FV — AdminSafe fallbackHandler can only be changed by Owners of AdminSafe or Modules of AdminSafe:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the fallbackHandler only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.AdminSafe.1, only Modules of AdminSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of AdminSafe as the threshold have approved (SE-5, SE-12). So only Modules of AdminSafe or Owners of AdminSafe can change the fallbackHandler.
P6.AdminSafe.3	`Modules of AdminSafe = null`, which can only be changed by Owners of AdminSafe because `Modules of AdminSafe = null`.	Same as P3.Admin.6. **FV — Modules of AdminSafe can only be changed by Owners of AdminSafe because `Modules of AdminSafe = null`:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the module list only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.AdminSafe.1, only Modules of AdminSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of AdminSafe as the threshold have approved (SE-5, SE-12). • The AdminSafe has no module (P3.Admin.6), so only the Owners of AdminSafe can change the module list.
P6.AdminSafe.4	`Owners of AdminSafe = Term multisig holders on the AdminSafe`, which can only be changed by Term	Same as P3.Admin.5.

#	Statement	Evidence
	multisig holders on the AdminSafe because `Modules of AdminSafe = null`.	**FV — Owners of AdminSafe can only be changed by Term multisig holders on the AdminSafe because `Modules of AdminSafe = null`:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the owners and threshold only change through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.AdminSafe.1, only Modules of AdminSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once four Owners of AdminSafe, the Term multisig holders (P3.Admin.5), have approved (SE-5, SE-12). • The AdminSafe has no module (P3.Admin.6), so only the Term multisig holders on the AdminSafe can change the owners and threshold.
P6.AdminSafe.5	AdminSafe guard is `address(0)`, which can only be changed by Owners of AdminSafe because `Modules of AdminSafe = null`.	**On-chain — AdminSafe guard is `address(0)`:** • ✅ At block 26229969, the guard slot (`0x4a204f620c8c5ccdca3fd54d003badd85ba500436a431f0cbda4f558c93c34c8`, `GuardManager.sol:30`) on the AdminSafe (`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`) is `address(0)`. **FV — AdminSafe guard can only be changed by Owners of AdminSafe because `Modules of AdminSafe = null`:** • SC-4 `settingsOnlyChangeThroughAModuleOrOwners` ✅ passes: over every write function, `fallback` included, the guard only changes through an enabled module's `execTransactionFromModule` or `execTransactionFromModuleReturnData`, or through `execTransaction`. • By P6.AdminSafe.1, only Modules of AdminSafe can call `execTransactionFromModule` and `execTransactionFromModuleReturnData` (SE-3), and `execTransaction` only acts once as many Owners of AdminSafe as the threshold have approved (SE-5, SE-12). • The AdminSafe has no module (P3.Admin.6), so only the Owners of AdminSafe can change the guard.
P6.AdminSafe.6	The owners, fallbackHandler, modules, guard for the AdminSafe can be changed by Term multisig holders on the AdminSafe.	**FV — The Term multisig holders on the AdminSafe can change the owners, fallbackHandler, modules and guard for the AdminSafe through `execTransaction`:** • SC-3 `ownersCanChangeSettings` ✅ passes: for each of `addOwnerWithThreshold`, `removeOwner`, `swapOwner`, `changeThreshold`, `enableModule`, `disableModule`, `setGuard` and `setFallbackHandler`, an owner's (Owners of Safe) `execTransaction`, addressed to the Safe with a call to that function, succeeds and changes a setting once the threshold is passed. • SE-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. The AdminSafe runs GnosisSafe v1.3.0 (P3.Admin.5), and its owners are the Term multisig holders on the AdminSafe (P3.Admin.5). • The AdminSafe has no guard (P6.AdminSafe.5), so nothing rejects the owners' `execTransaction`.

PauseGuard

P6.PauseGuard: The pauser for the PauseGuard can only be changed by Term multisig holders on the AdminSafe; the ADMIN_ROLE holder for the PauseGuard cannot be changed by anyone.

#	Statement	Evidence
P6.PauseGuard.1	Only the PauseGuard pauser and the PauseGuard ADMIN_ROLE holder can interact with the PauseGuard, except `checkTransaction` and `checkAfterExecution`, which anyone can call but which do not change state, and except `grantRole`, `revokeRole` and `renounceRole`, which revert for every caller.	Same as P3.1.

#	Statement	Evidence
P6.PauseGuard.2	`PauseGuard ADMIN_ROLE holder = AdminSafe`, which cannot be changed by anyone.	Same as P3.Admin.2. **FV — Nobody can change the PauseGuard ADMIN_ROLE holder:** • DP-15 `adminRoleNeverChanges` ✅ passes: over every write function of the Delay Modifier and the PauseGuard, ADMIN_ROLE membership never changes for any account. • DP-12 `renounceRoleAlwaysReverts` ✅ passes: no holder can drop a role. • DP-13 `inheritedGrantAndRevokeAlwaysRevert` ✅ passes: `grantRole` and `revokeRole` revert for every role and every caller, admins included, because both roles are administered by `DEFAULT_ADMIN_ROLE`, which nobody holds. • PD-1 `defaultAdminRoleNeverHeld` ✅ passes: from the constructor on, nobody holds `DEFAULT_ADMIN_ROLE`. • DP-16 `defaultAdminRoleNeverGranted` ✅ passes: an unheld `DEFAULT_ADMIN_ROLE` stays unheld. • PD-2 `roleAdminWiringIsFixed` ✅ passes: from the constructor on, both ADMIN_ROLE and `DEFAULT_ADMIN_ROLE` are administered by `DEFAULT_ADMIN_ROLE`. • DP-17 `roleAdminWiringNeverChanges` ✅ passes: nothing re-points the admin of ADMIN_ROLE or `DEFAULT_ADMIN_ROLE`. • PD-3 `exactlyOneAdminRoleHolder` ✅ passes: from the constructor on, ADMIN_ROLE has exactly one holder, the AdminSafe (P3.Admin.2).
P6.PauseGuard.3	`PauseGuard pauser = PauseSafe`, which can only be changed by Term multisig holders on the AdminSafe.	Same as P3.Pause.2. **FV — The PauseGuard pauser can only be changed by Term multisig holders on the AdminSafe:** • DP-14 `pauserOnlyChangesThroughSetPauser` ✅ passes: over every write function of both contracts, the pauser only changes through `setPauser`. • DP-10 `setPauserRevertsWithoutAdminRole` ✅ passes: without ADMIN_ROLE, `setPauser` reverts. • The only ADMIN_ROLE holder is the AdminSafe (P6.PauseGuard.2), and only the Term multisig holders on the AdminSafe can make it act (P6.AdminSafe.1, P6.AdminSafe.3, P6.AdminSafe.4). So only they can change the pauser.
P6.PauseGuard.4	The pauser for the PauseGuard can be changed by Term multisig holders on the AdminSafe; the ADMIN_ROLE holder for the PauseGuard cannot be changed by Term multisig holders on the AdminSafe, so it cannot be changed by anyone.	**FV — The Term multisig holders on the AdminSafe can change the PauseGuard pauser:** • DP-11 `setPauserSetsThePauser` ✅ passes: for any nonzero account, an ADMIN_ROLE holder's `setPauser(account)` succeeds, `pauser()` then returns that account, and the previous pauser no longer holds the slot. • SE-6 `ownersCanMakeTheSafeAct` ✅ passes: once the threshold is passed, `execTransaction` succeeds and the Safe acts, for any threshold and any number of owners. So the AdminSafe's owners (P3.Admin.5) can have it call `setPauser`. **FV — The Term multisig holders on the AdminSafe cannot change the PauseGuard ADMIN_ROLE holder, so nobody can:** • Same as P6.PauseGuard.2: ADMIN_ROLE membership never changes, whoever the caller is, the AdminSafe included.

4. Appendix

4.1 Formal Verification

All Certora specs and confs for this document: `packages/evm/certora/2.0.0`.

Each rule cited in Section 3 is named by a prefix and a number, such as SE-5 or DM-13. The table below lists each prefix, what its rules cover, which of its rules Section 3 uses, the spec that defines them and the conf that runs them. A prefix whose rules are spread over more than one spec has one row per spec.

Prefix	What it covers	Rules used in Section 3	Spec	Conf
SE	GnosisSafe v1.3.0 entry points and who can make the Safe act	SE-1, SE-3 to SE-12	`Safe/executionPaths.spec`	`Safe-executionPaths.conf`
	GnosisSafe v1.3.0 `fallback` and `receive` change no state	SE-21 to SE-26	`Safe/noStateChange.spec`	`Safe-noStateChange.conf`
SC	GnosisSafe v1.3.0 settings change only through a module or the owners	SC-1 to SC-4	`Safe/selfCalls.spec`	`Safe-selfCalls.conf`
SE141	GnosisSafe v1.4.1 entry points and who can make the Safe act	SE141-1, SE141-3 to SE141-9, SE141-11, SE141-12	`SafeV141/executionPaths.spec`	`Safe-v141-executionPaths.conf`
	GnosisSafe v1.4.1 `fallback` and `receive` change no state	SE141-21 to SE141-26	`SafeV141/noStateChange.spec`	`Safe-v141-noStateChange.conf`
SC141	GnosisSafe v1.4.1 settings change only through a module or the owners	SC141-1 to SC141-4	`SafeV141/selfCalls.spec`	`Safe-v141-selfCalls.conf`
CL	ConfigLockGuard on a GnosisSafe v1.3.0 Safe: what owners cannot change, and what owners and modules can still do	CL-1 to CL-5, CL-8 to CL-20	`Safe/configLockGuard.spec`	`Safe-configLockGuard.conf`
LG	ConfigLockGuard on the OwnerlessSafe: the module path	LG-2, LG-3	`Safe/ownerlessSafeGuard.spec`	`Safe-ownerlessSafeGuard.conf`
CL141	ConfigLockGuard on a GnosisSafe v1.4.1 Safe: what owners cannot change on the Safe and its locked modifier, and what owners can still do	CL141-1 to CL141-10, CL141-12 to CL141-19, CL141-21, CL141-23	`SafeV141/configLockGuard.spec`	`Safe-v141-configLockGuard.conf`
OD	ConfigLockGuard locking the Delay Modifier: no unlocked call changes its owner, target, modules or guard, and the DelayOwnerSafe can still veto and set txCooldown and txExpiration	OD-1, OD-2, OD-4 to OD-6	`Delay/delayOwnerGuard.spec`	`Delay-delayOwnerGuard.conf`
DM	Delay Modifier access control and module list	DM-1, DM-2, DM-4 to DM-8, DM-11 to DM-13, DM-15 to DM-20	`Delay/moduleIntegrity.spec`	`Delay-moduleIntegrity.conf`
DQ	Delay Modifier queue provenance	DQ-2 to DQ-6	`Delay/queueProvenance.spec`	`Delay-queueProvenance.conf`
DE	DEVOPS_ROLE methods end to end	DE-1, DE-2	`Delay/devopsEndToEnd.spec`	`Delay-devopsEndToEnd.conf`
DV	DelayOwnerSafe owners veto directly	DV-1	`Delay/vetoSignedPath.spec`	`Delay-vetoSignedPath.conf`
DP	PauseGuard, and the Delay Modifier's veto window and vetoes	DP-3 to DP-21, DP-27 to DP-35	`Delay/pauseGuardSufficient.spec`	`Delay-pauseGuardSufficient.conf`
PD	PauseGuard deployment: what its constructor sets up holds for the life of the contract	PD-1 to PD-3	`PauseGuard/deployment.spec`	`PauseGuard-deployment.conf`
PO	ProposerSafe → Delay Modifier → OwnerlessSafe settings	PO-5	`Delay/proposerToOwnerlessSettings.spec`	`Delay-proposerToOwnerlessSettings.conf`
RI	Roles Modifier configuration integrity	RI-1, RI-2, RI-5 to RI-16, RI-19, RI-20, RI-22 to RI-24	`Roles/configIntegrity.spec`	`Roles-configIntegrity.conf`
RG	Governor limited to `Delay.setTxNonce`	RG-2 to RG-5	`Roles/setTxNonceGuardAndRoleConfig.spec`	`Roles-setTxNonceGuardAndRoleConfig.conf`
GP	Governor proposal threshold	GP-1	`Governor/proposalThreshold.spec`	`Governor-proposalThreshold.conf`
GS	Governor entry points and settings	GS-1 to GS-18	`Governor/settings.spec`	`Governor-settings.conf`
GC	Governor vote counting	GC-1 to GC-3, GC-5	`Governor/voting.spec`	`Governor-voting.conf`

Prefix	What it covers	Rules used in Section 3	Spec	Conf
	Only votes move a tally	GC-4	`Governor/tallies.spec`	`Governor-tallies.conf`
GV	Governor veto	GV-1, GV-2, GV-4	`Governor/veto.spec`	`Governor-veto.conf`
	The veto lands on the DelayOwnerSafe (GnosisSafe v1.4.1)	GV-3	`SafeV141/vetoLandsOnDelay.spec`	`Safe-v141-vetoLandsOnDelay.conf`
GE	Veto end to end	GE-1	`Governor/vetoEndToEnd.spec`	`Governor-vetoEndToEnd.conf`
TT	TERM token: total supply, minting, burning, upgrades and total-supply checkpoints	TT-1, TT-2, TT-4 to TT-17	`TermToken/supply.spec`	`TermToken-supply.conf`

4.2 On-chain

All contracts are on Ethereum mainnet. Each address links to its Etherscan page.

Safes

Contract	Address
OwnerlessSafe	`0xb8A1dF43c1c88b13937C0c5CEBbAd15830cAeC03`
ProposerSafe	`0xd5E12854A3Dba99deF295A7635D3Ba16427d2A28`
DelayOwnerSafe	`0x2a875746D0c88EBD2bbBfc8F8a773c58c3373ad3`
PauseSafe	`0x74f3F3dEfdC563bbFC8637BaB2d30596D2817472`
AdminSafe	`0x73d1C7dc9CEb14660Cf1E9BB29F80ECF9E97D774`
GnosisSafe v1.3.0 mastercopy	`0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552`

Modifiers

Contract	Address
Delay Modifier	`0x0C19d8A404079d71E5CA3e32fE3f758Ab543ACdf`
Roles Modifier	`0xaBAC51B6AEb05a2CE65310F79e64DF203D6c8Ab3`

Guards

Contract	Address
PauseGuard	`0x3A70244c0fEc95B5Dd238dfa60DAbDB60a99f618`
SetTxNonceGuard	`0x7aE03372ECDcEe335CdEfB7d354d507F0506616C`

Governance

Contract	Address
Governor	`0x2B715634134220ffeEE9458b4e34E41A41418607`
TERM token	`0xC3d21f79C3120A4fFda7A535f8005a7c297799bF`

Term Protocol contracts

Contract	Address
TermController	`0xd902EBb8AEb832643af38f43d466a6155cD8Bd5A`
TermEventEmitter	`0xf021B31282a60528B2F47D07Ce353da870be78b3`
TermPriceConsumerV3	`0x13Ca4ddB295d621761057D682a1c7b5f5D7Bba4c`
TermRepoDeployerFactory	`0x44058C32B154A516F3b9DE2413e6cb938F93c3B4`
TermInitializer	`0x3B668F408293e9a5700D6Eeae78E1f945CF501eF`
TermDiamond	`0xb4B1de03F229220eDbBB31A224b2a004b3c3e47f`